

Informations verarbeitung

Machine Learning Hardware und Systeme

Dr.-Ing. Tanja Harbaum

Institut für Technik der Informationsverarbeitung (ITIV)

Anmeldung Prüfung

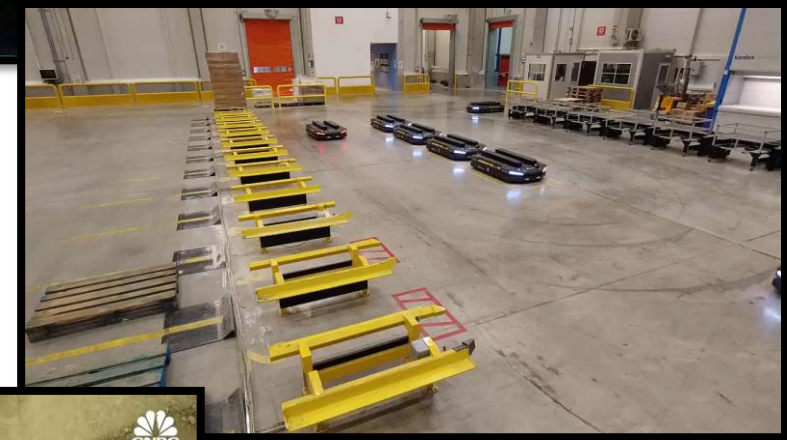
- Workshop: Anmeldung in CAS möglich bis zum 17.07.
- Workshop ist Voraussetzung für die Zulassung der schriftlichen Prüfung
 - Anmeldung bis zum 28.07. möglich
 - Prüfung am 04.08.
- Abschließen des Workshops muss bis zum 20.07. 14:00 Uhr geschehen

Übersicht – Vorlesung Nr. 04

- Einführung neuronale Netze
- Was berechnen neuronale Netze?
 - Dense Neural Networks
 - Convolutional Neural Networks
- Ausführung neuronaler Netze auf Hardware
 - Prinzipien der Beschleunigung von neuronalen Netzen
 - Systolische Arrays
 - In / Near-Memory Computing

Motivation

- Heute sind KI-Anwendungen nicht mehr weg zu denken
- Klassische Anwendungen
 - Robotik
 - Autonome Fahrzeuge
 - Automatisierung
 - Logistik



Machine Learning – Fundamentals

“

A computer program is said to learn from experience (E) with respect to some task (T) and some performance measure (P), if its performance on T, as measured by P, improves with experience E.

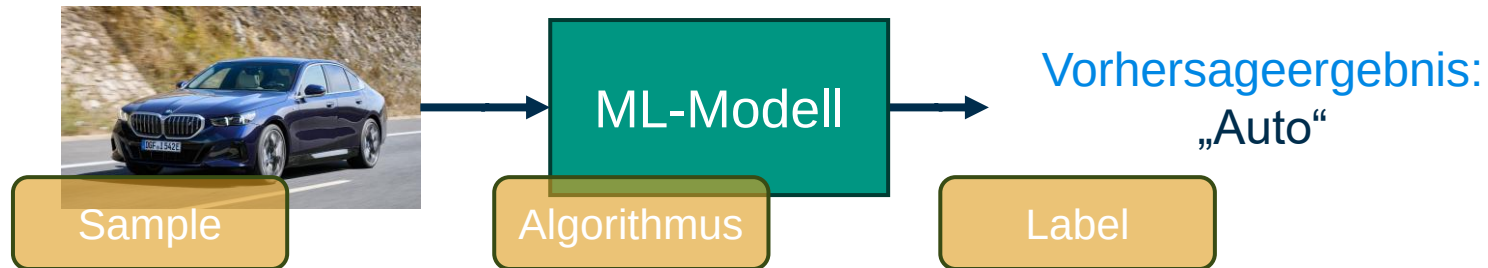
Tom M. Mitchell, Carnegie Mellon University (1998)

”

- Was sind E, T und P?
 - E: Empirische Werte → Eingabedaten (samples)
 - Alle Samples formen zusammen den **Datensatz**
 - T: Aufgabe → Was soll der Algorithmus vorhersagen?
 - P: Bewertungsmetrik → Woran erkennt der Algorithmus, dass er „richtig“ trainiert wurde und „das Richtige“ vorhersagt?

Grundlagen – Machine Learning – Datensatz (E)

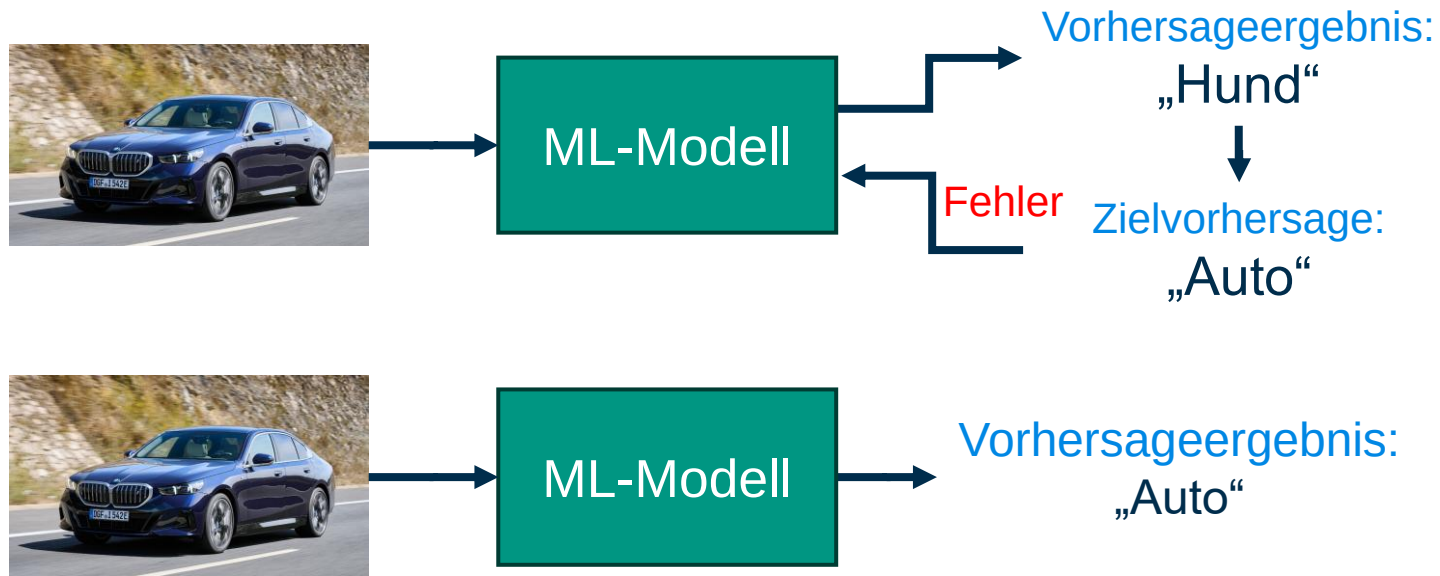
- Bestehen aus Eingabedaten (Samples), die individuelle Eigenschaften haben (Features)
- Optional: „gewünschte“ Ausgabedaten (Labels)
 - Bsp.: Klassifikationsdatensatz:



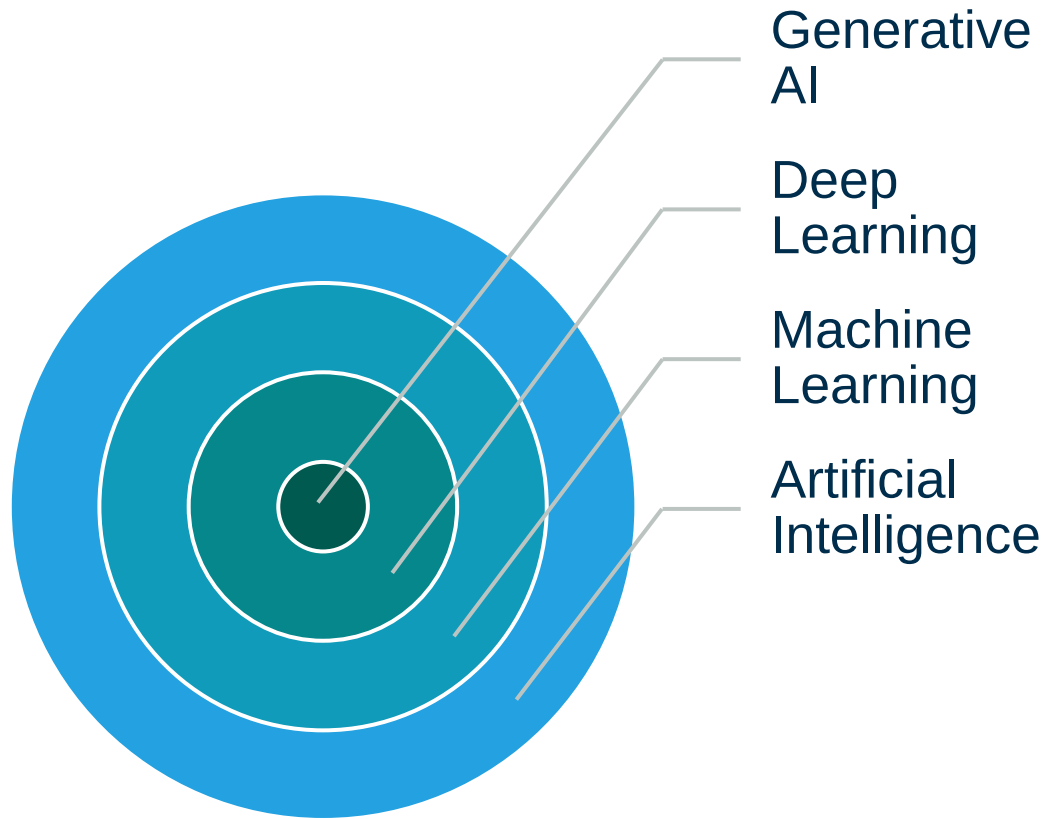
Grundlagen – Machine Learning

■ Training vs. Inferenz

- Ein Modell muss zunächst trainiert werden damit es danach auf ein Zielsystem ausgerollt werden kann
- Die „Ausführung“ des Modells ohne Anpassung der Parameter ist die Inferenz

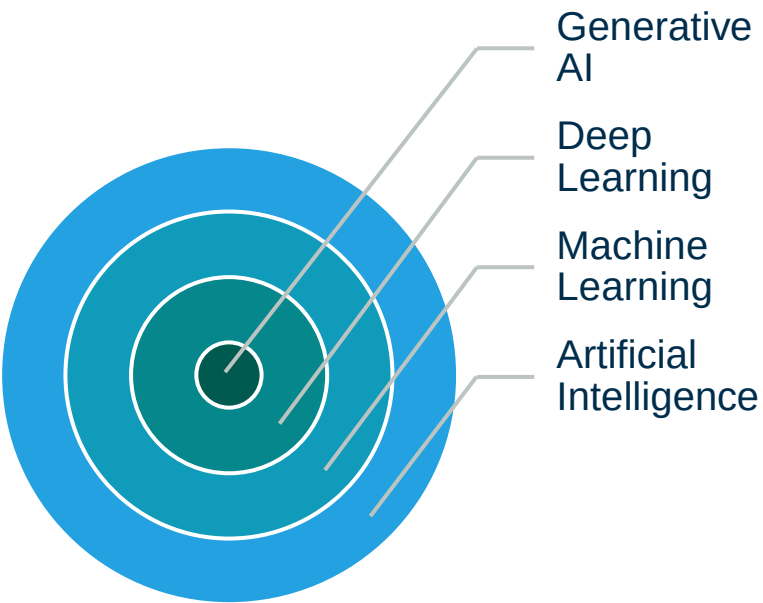


Grundlagen – Machine Learning

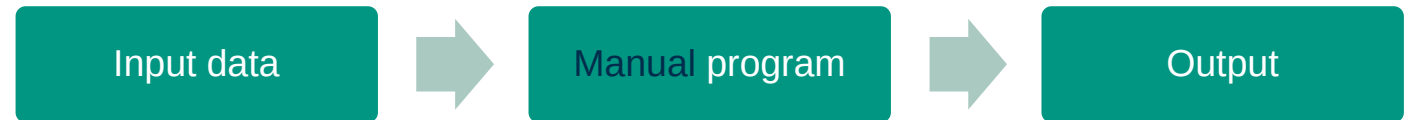


- Künstliche Intelligenz ist der „Marketing“ Überbegriff
- Machine Learning (ML) ist Überbegriff für selbstlernende Systeme
 - Systeme, die aus Daten lernen
- Deep Learning bezeichnet besonders komplexe selbstlernende Systeme
 - z.B. Neuronale Netze (NN)
- Generative KI-Modelle sind riesige tiefe neuronale Netze, die Inhalte generieren können
 - Z.B. LLMs

Grundlagen – Machine Learning



Klassischer Algorithmus



Klassischer ML Algorithmus



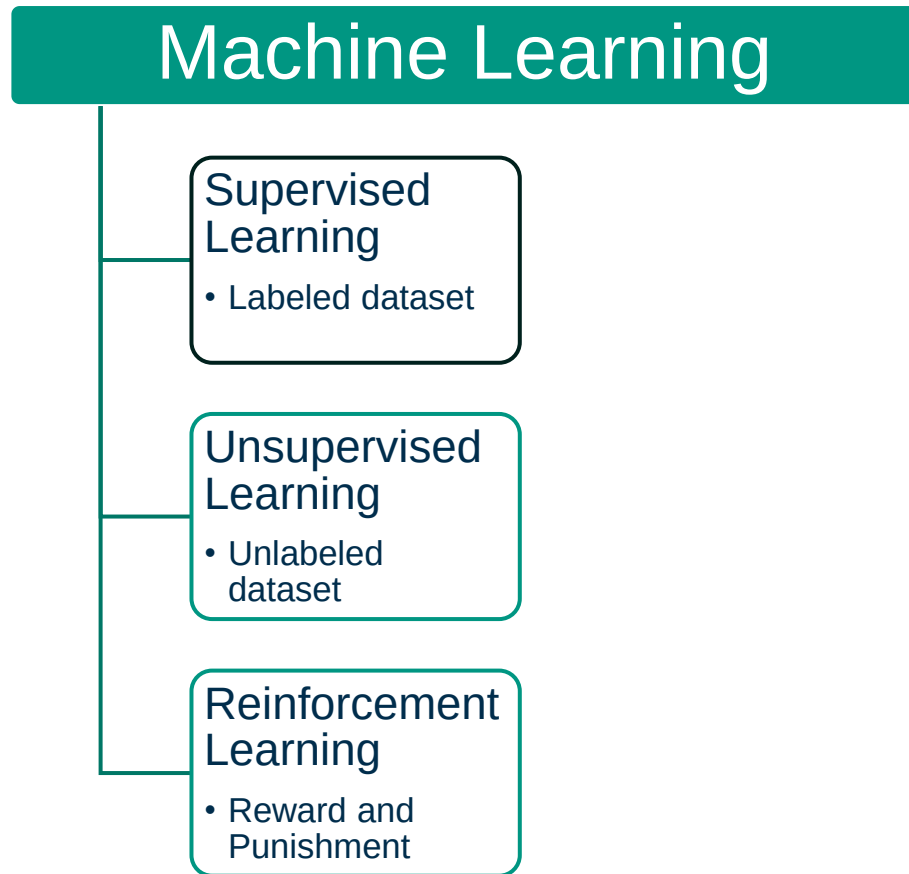
Deep Learning



Generative AI



Grundlagen – Machine Learning Training



- **Supervised Learning**
 - Der Algorithmus verarbeitet unzählige Eingabedatenpunkte zusammen mit den erwarteten Vorhersagen (Labels oder Ground Truths)
- **Unsupervised Learning**
 - Modell versucht anhand von Metriken die Daten zu strukturieren (Bsp Clustering)
- **Reinforcement Learning**
 - "Agent" lernt durch Versuche (Trial-and-Error) in einer Umgebung, um optimale Entscheidungen zu treffen



Sie haben einen Roboterarm der für Sie Pfannkuchen wenden soll. Welche Lernmethode eignet sich am besten?



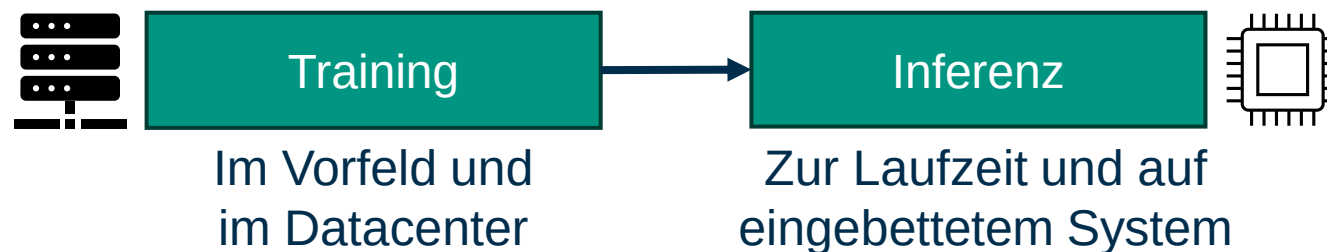
Sie haben Bilder von Haustieren und entsprechende Beschriftungen für jedes Bild. Ihr Modell soll Katzen erkennen. Welche Lernmethode eignet sich am besten?



Sie haben einen Datensatz an Sensorwerten von industriellen Maschine. Sie wollen auf Basis neuer Sensorwerte vorhersagen, ob ein Defekt vorliegt. Welche Lernmethode eignet sich am besten?

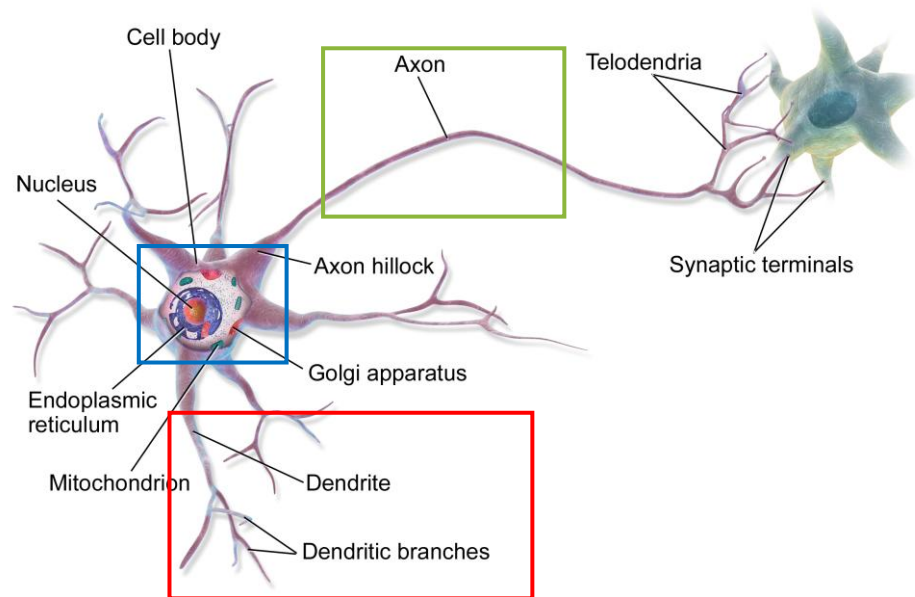
Grundlagen – Machine Learning Training

- Fokus in IV: Supervised Learning am Beispiel von neuronalen Netzen
- Hardwareaspekt:
 - Training ist tendenziell sehr rechenintensiv (unzählige Samples müssen ausgewertet werden) → Ausführung im Datacenter
- Inferenz ist die anschließende Ausführung auf einer Zielhardware, z.B. einem eingebetteten System
 - Auswertung einzelner Samples → weniger rechenintensiv

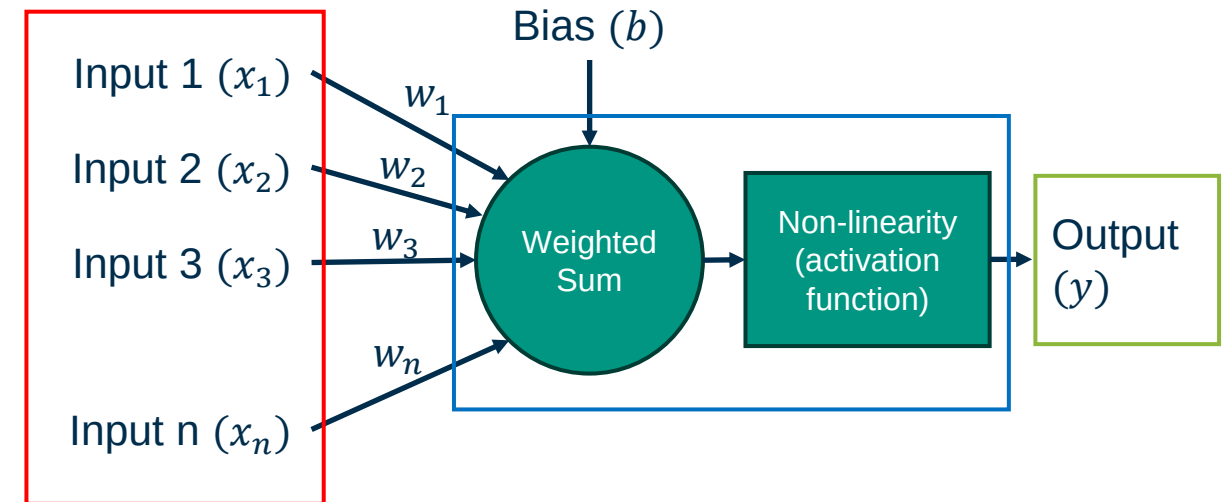


Grundlagen – Neuronale Netze (NN)

- Inspiriert an biologischen neuronalen Netzen



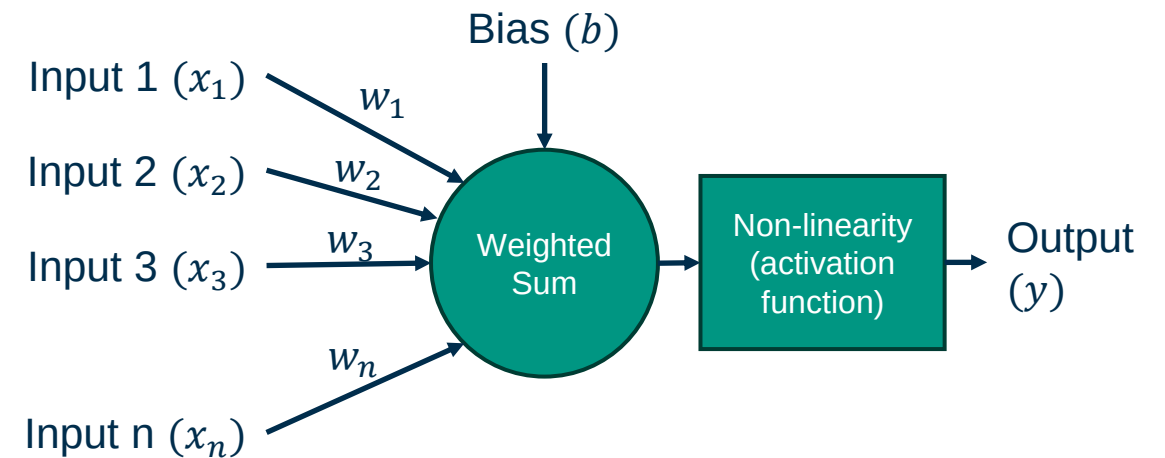
Wikipedia – Neuron CC BY 3.0



$$b + \sum_{i=1}^n x_i * w_i$$

Grundlagen – Neuronale Netze (NN)

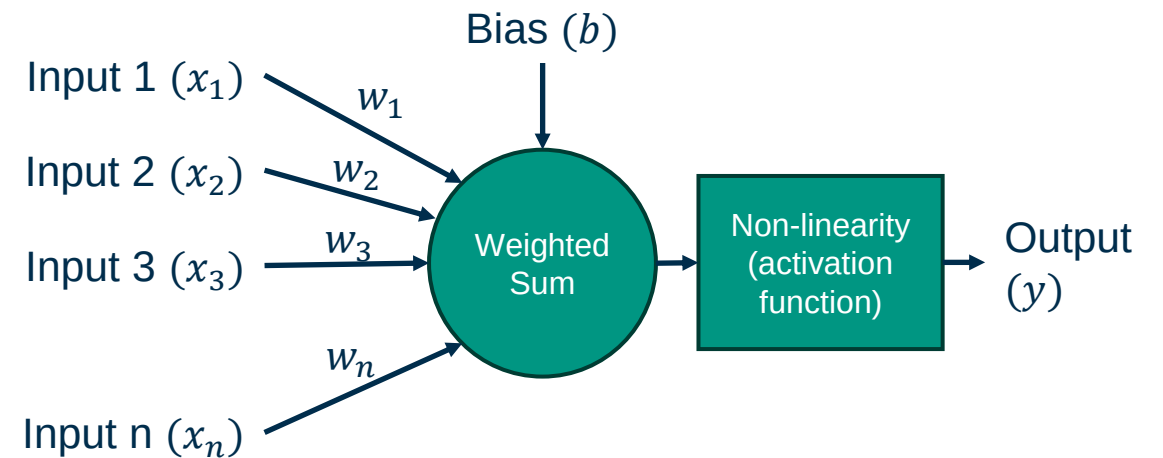
- b Bias (Verzerrung)
- x Input in das Neuron
- w Gewichte
- n Anzahl von Inputs aus der vorhergehenden Schicht



$$y = \sigma \left(b + \sum_{i=1}^n x_i * w_i \right)$$

Grundlagen – Neuronale Netze (NN)

- Inputs sind z. B. die Merkmale des Datensatzes
- Gewichte und Bias werden so gelernt, dass die Ausgabe der Klassifizierung entspricht
- Aktivierungsfunktion, um nichtlineare Beziehungen abbilden zu können

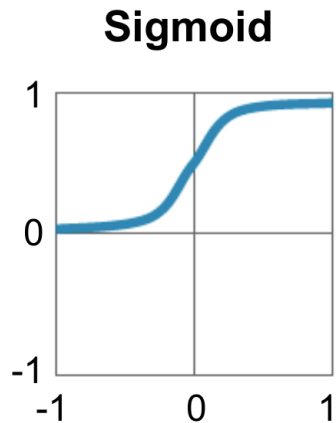


$$y = \sigma \left(b + \sum_{i=1}^n x_i * w_i \right)$$

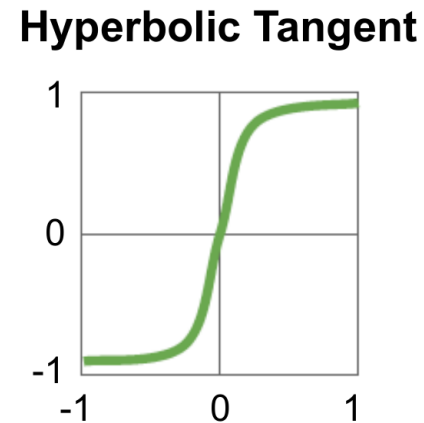
Grundlagen – Neuronale Netze (NN)

- Aktivierungsfunktionen für die nicht-linearität

- Traditionelle



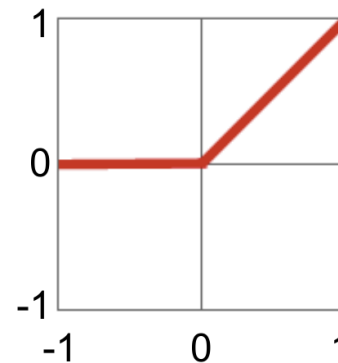
$$y = 1 / (1 + e^{-x})$$



$$y = (e^x - e^{-x}) / (e^x + e^{-x})$$

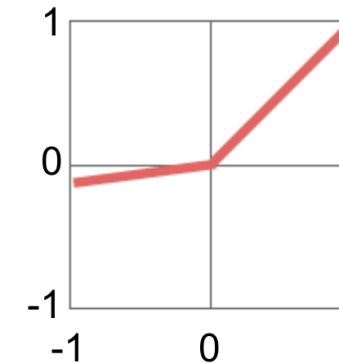
- Modernere

Rectified Linear Unit (ReLU)



$$y = \max(0, x)$$

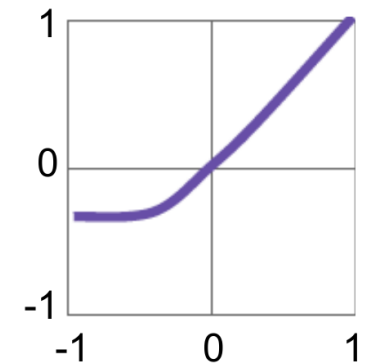
Leaky ReLU



$$y = \max(\alpha x, x)$$

α = small const. (e.g. 0.1)

Exponential LU

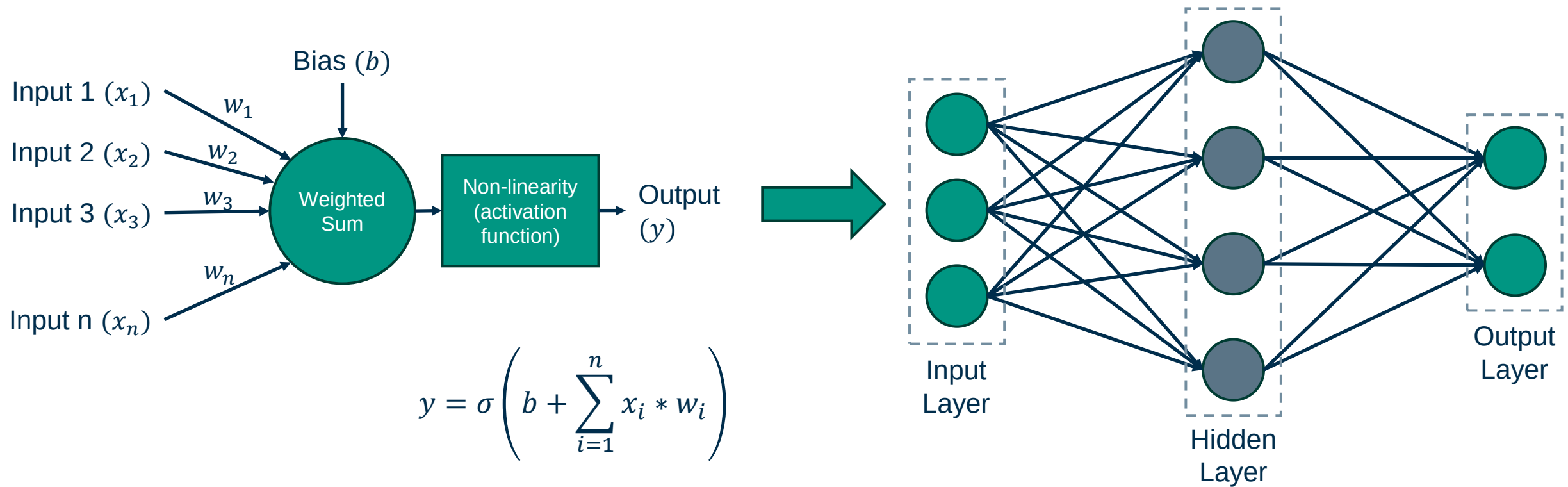


$$y = \begin{cases} x, & x \geq 0 \\ \alpha(e^x - 1), & x < 0 \end{cases}$$

HW for ML, Shao et al. 2021

Grundlagen – Neuronale Netze (NN)

- Mehrere künstliche Neuronen formen dann ein neuronales Netzwerk



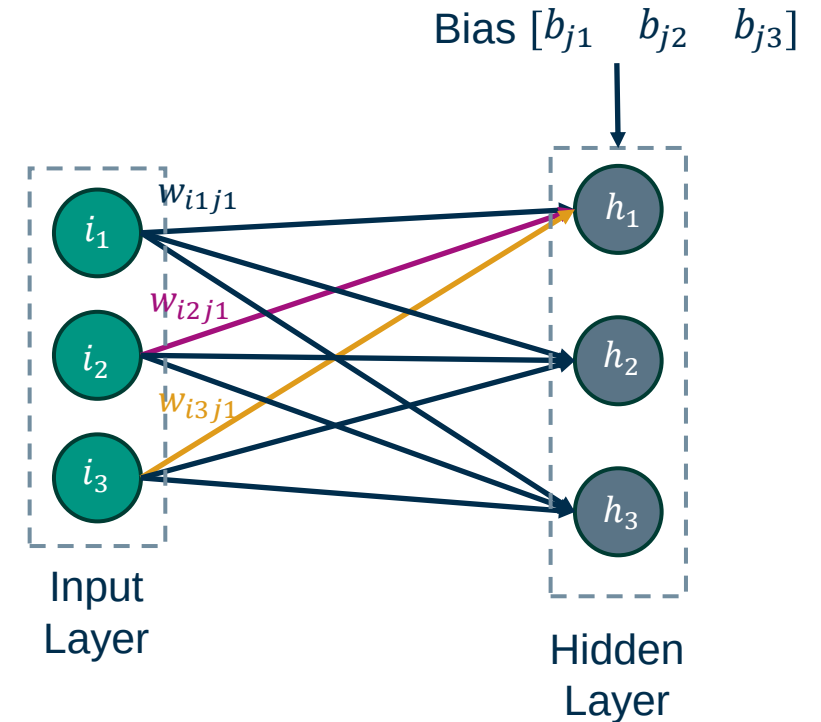
Grundlagen – Neuronale Netze (NN)

- Diese tiefen neuronalen Netze (DNN) können anspruchsvolle Aufgaben wie die Bildverarbeitung bewältigen
- Beispiel: Berechnung einer einzelnen Schicht mit 3 Neuronen:

$$\begin{bmatrix} i_1 & i_2 & i_3 \end{bmatrix} \times \begin{bmatrix} w_{i1j1} & w_{i1j2} & w_{i1j3} \\ w_{i2j1} & w_{i2j2} & w_{i2j3} \\ w_{i3j1} & w_{i3j2} & w_{i3j3} \end{bmatrix} + \begin{bmatrix} b_{j1} & b_{j2} & b_{j3} \end{bmatrix} = \begin{bmatrix} h_1 & h_2 & h_3 \end{bmatrix}$$

- Generell:

$$\text{inputs} \times \text{weights} + \text{bias} = \text{outputs}$$





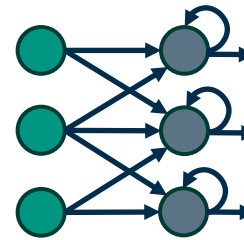
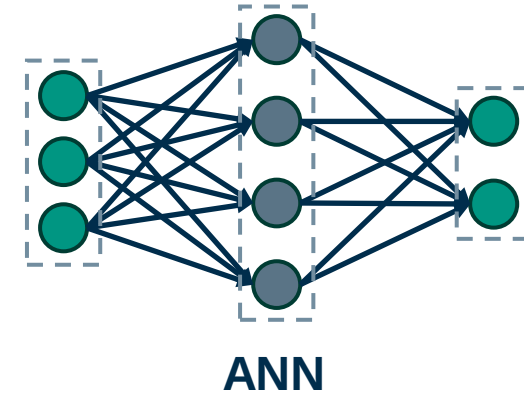
Eine Schicht Ihres Fully Connected NNs bekommt 10 Eingabewerte und enthält 20 Neuronen. Wie viele Gewichte hat diese Schicht OHNE Bias zu zählen?



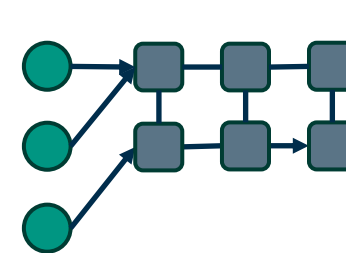
Eine Schicht Ihres Fully Connected NNs bekommt 10 Eingabewerte und enthält 20 Neuronen. Wie viele Bias Gewichte (/Parameter) hat diese Schicht?

Grundlagen – verschiedene Neuronale Netze

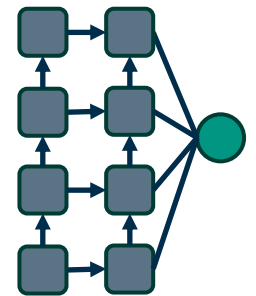
- Bisher haben wir uns mit sogenannten dichten oder vollständig verbundenen Netzwerken befasst
- Aber es gibt auch noch
 - Convolutional Neural Networks (CNN)
 - Recurrent Neural Networks (RNNs)
 - Long-Short-Term Memories (LSTM)
 - Transformer-Modelle
 - z. B. Generative Pretrained Transformer (GPT)



RNN



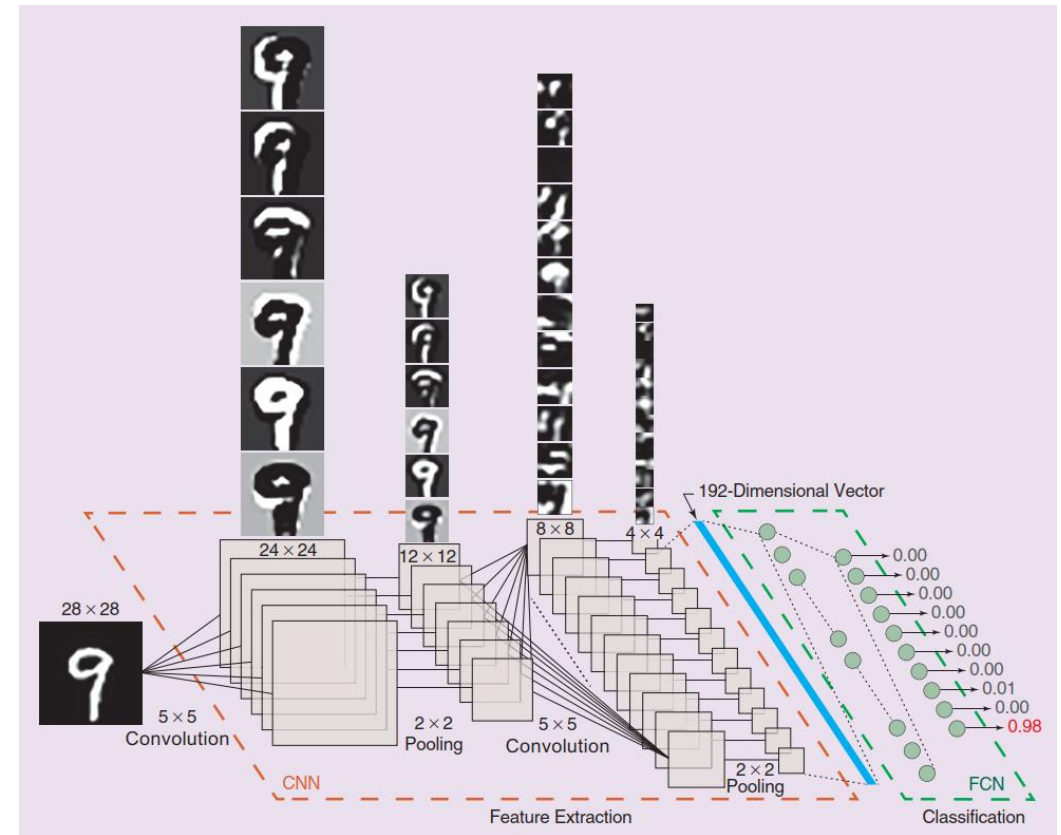
CNN



Transformer

Convolutional Neural Networks (CNNs) – Intro

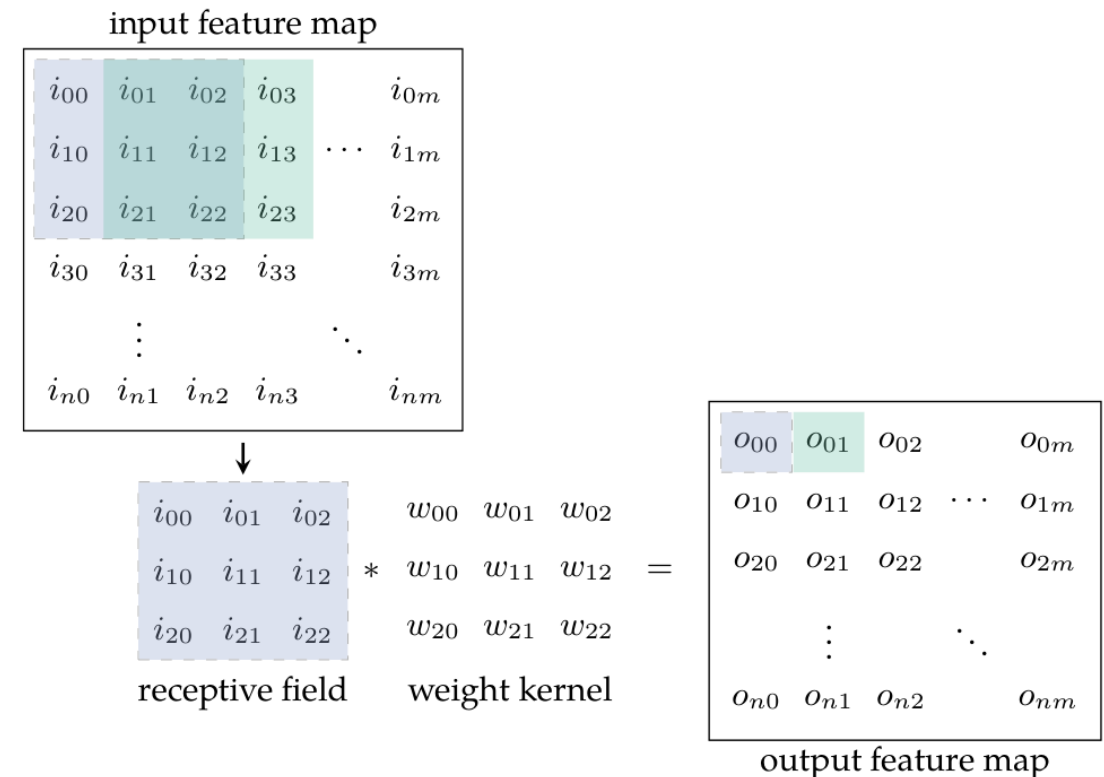
- Entwickelt zur Verarbeitung von Daten in einer 2D-Topologie (Text, Bilder, Audio usw.) und zur Erstellung von Vorhersagen
 - Weit verbreitet in der Bildverarbeitung und im Bereich Computer Vision
- Die Faltung ist die wichtigste Operation zur Merkmalsextraktion
 - Mehrere Stufen von Faltung und Pooling zum Erlernen komplexer Muster
- Die extrahierten Merkmale werden verwendet, um die Ausgabeklasse durch ein vollständig verbundenes Netzwerk (FCN) vorherzusagen



Source: R. C. Gonzalez, "Deep Convolutional Neural Networks [Lecture Notes]," IEEE Signal Process. Mag., vol. 35, no. 6, pp. 79–87, Nov. 2018.

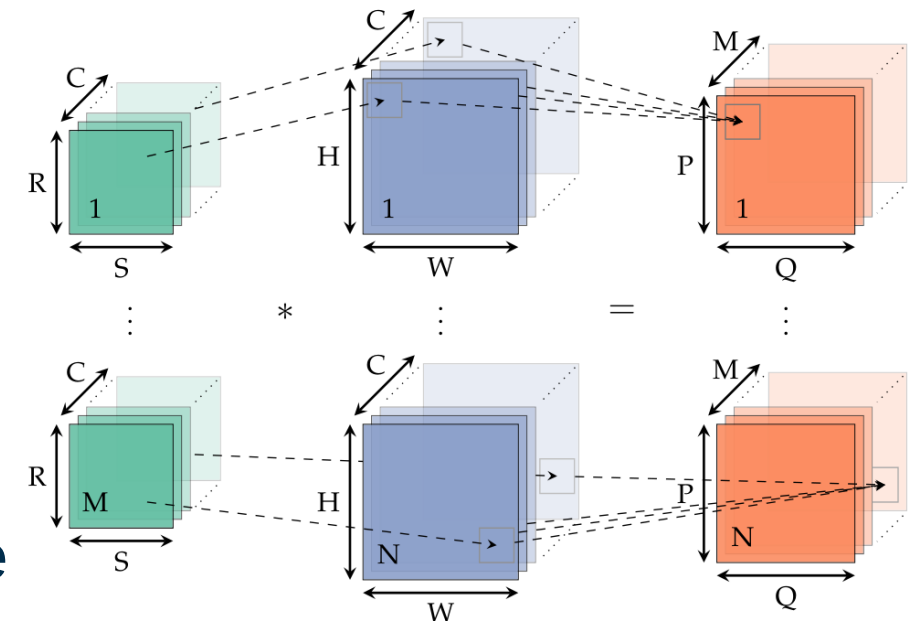
Grundlagen – Convolutional Neural Networks

- CNNs extrahieren räumlich zusammenhängende Informationen mit sogenannten convolution-Filtern (engl. Convolution = Faltung)
- Eingangsdaten (input feature map) wird mit einem Kernel aus vorher trainierten Gewichten gefaltet und erzeugt die Ausgangsdaten (output feature map), auf die danach auch die Aktivierungsfunktion angewandt wird



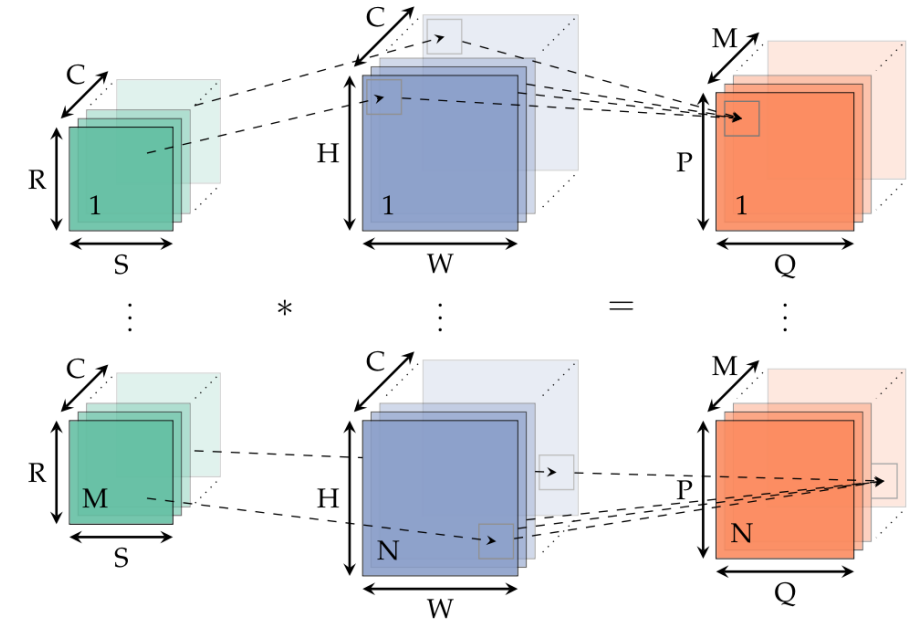
Grundlagen – Convolutional Neural Networks (CNNs)

- In der Praxis ist ein CNN-Layer mehrdimensional
- Eingangsdaten sind nicht nur zwei-dimensional, sondern haben mehrere Kanäle
- Jeder Filter besteht aus einer Reihe von Kernels
 - Einem für jeden Eingangskanal
 - Die Kernel werden mit den entsprechenden Eingangskanälen gefaltet, und die Ergebnisse werden summiert
- Pro Schicht gibt es mehrere Kernel, die unterschiedliche Arten von Informationen extrahieren
 - Viele Kernel zusammen bilden einen Faltungsfilter
- Jeder Filter erzeugt einen Ausgangskanal



Grundlagen – Convolutional Neural Networks (CNNs)

- N Batch size des Inputs
 - Bleibt im gesamten Netzwerk unverändert
- C Kanäle – Tiefe des Inputs
- M Anzahl der Filter
 - Legt die Anzahl der Kanäle in der Output-Feature-Map fest
- W / H Breite und Höhe des Inputs
- R / S Breite und Höhe eines Filters
 - als Kernelgröße bezeichnet
- P / Q Breite und Höhe des Outputs
- U Schrittweite der Faltung



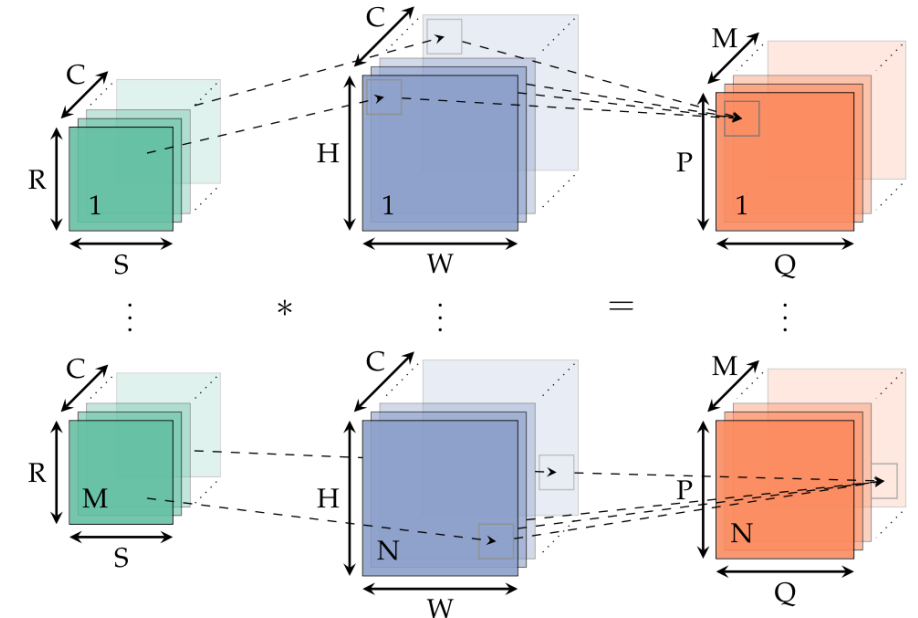
$$o[n][m][p][q] = \sigma\left(\sum_{c=0}^{C-1} \sum_{r=0}^{R-1} \sum_{s=0}^{S-1} i[n][c][Up+r][Uq+s] * w[m][c][r][s]\right) + b[m]$$

Grundlagen – Convolutional Neural Networks (CNNs)

$$o[n][m][p][q] = \sigma \left(\sum_{c=0}^{C-1} \sum_{r=0}^{R-1} \sum_{s=0}^{S-1} i[n][c][Up+r][Uq+s] * w[m][c][r][s] \right) + b[m]$$

```

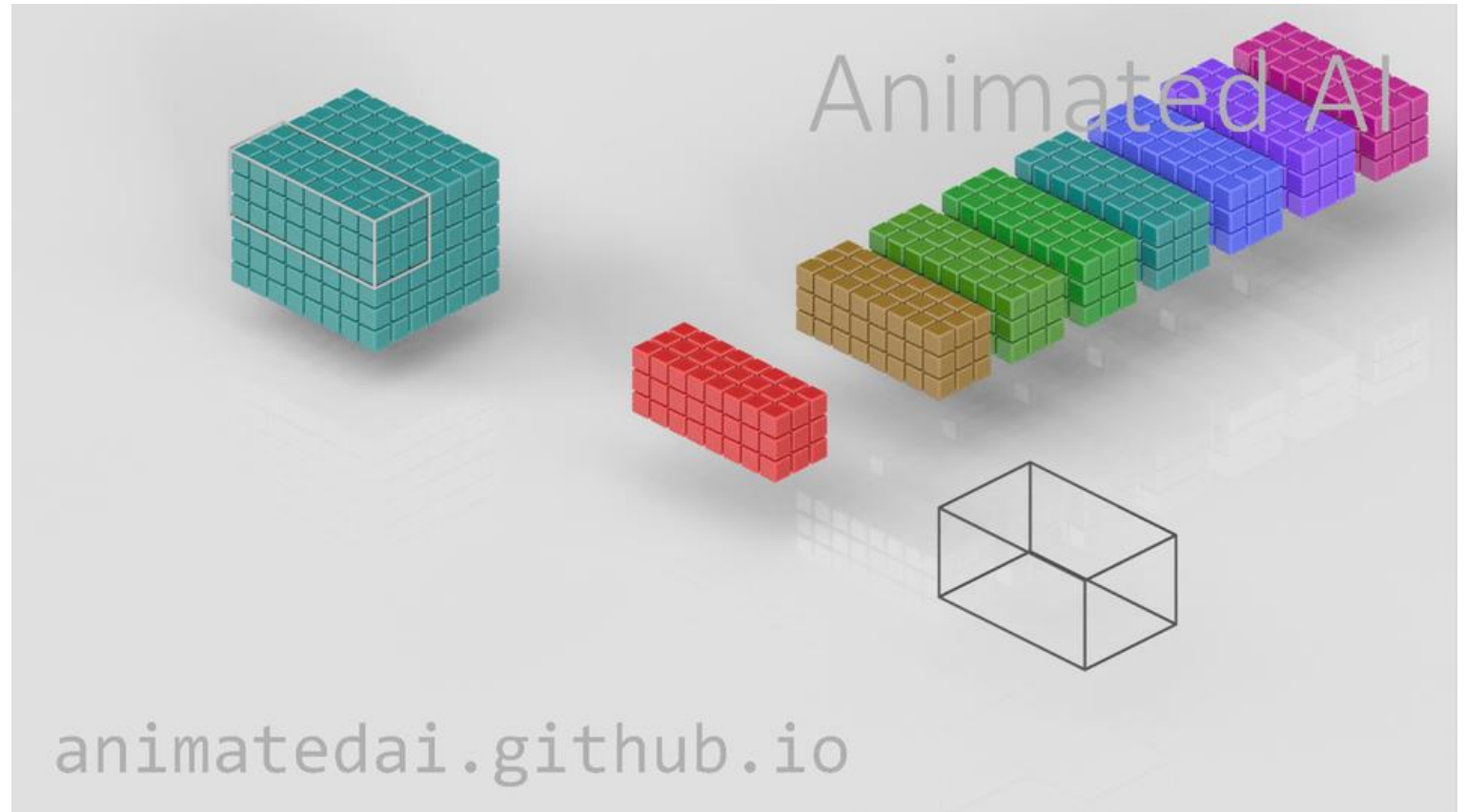
for (n=0; n<N; n++) {
  for (m=0; m<M; m++) {
    for (p=0; p<P; p++) {
      for (q=0; q<Q; q++) {
        O[n][m][p][q] = Bias[m];
        for (r=0; r<R; r++) {
          for (s=0; s<S; s++) {
            for (c=0; c<C; c++) {
              O[n][m][p][q] += I[n][c][Up+r][Uq+s]
                             * W[m][k][r][s];
            }
          }
        }
        O[n][m][p][q] = Activation(O[n][m][p][q]);
      }
    }
  }
}
    
```



7 verschachtelte Schleifen!

Convolutional Neural Networks (CNNs)

- $C = 8$ Kanäle
 - Tiefe des Inputs
- $M = 8$ Anzahl von Filtern
- $W = 7 / H = 6$ Input Breite und Höhe
- $R = 3 / S = 3$ Breite und Höhe eines Filters
- $P = 5 / Q = 4$ Breite und Höhe des Outputs



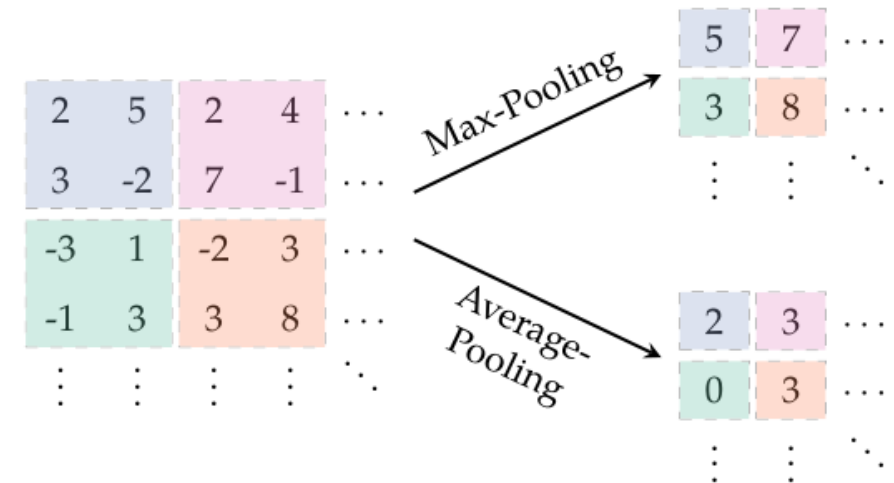
Source: <https://animatedai.github.io/media/convolution-animation-3x3-kernel.gif>



Ein Bild hat die Maße $W * H$, eine Schicht in Ihrem CNN hat 4 Filter der Größe 3×3 . Wie groß ist dann die Ausgabe dieser Schicht?

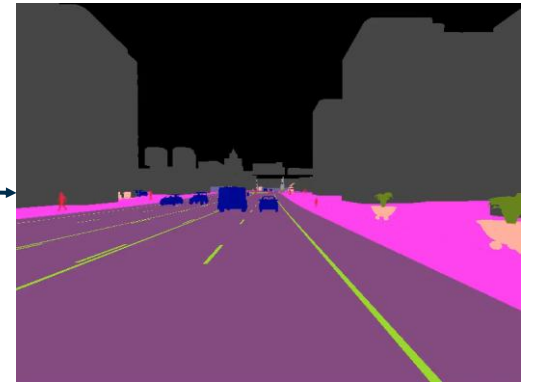
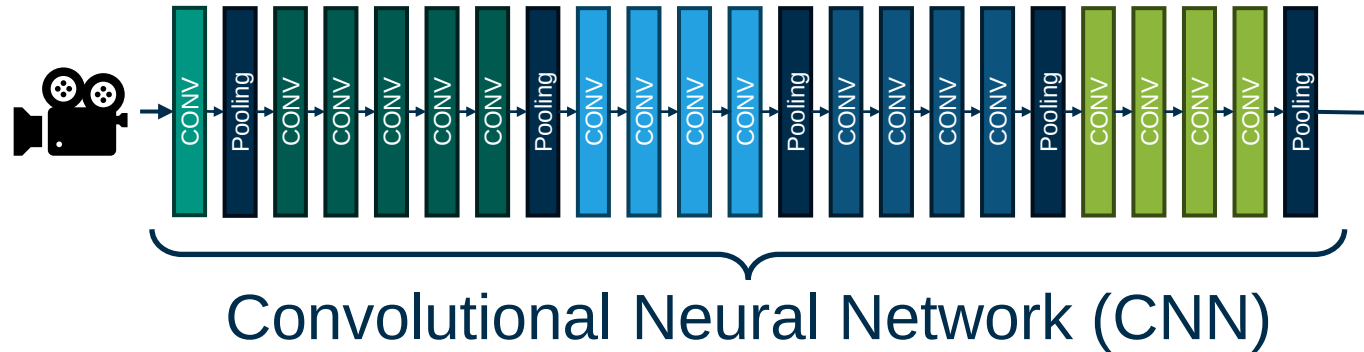
Grundlagen – CNNs – Pooling

- Pooling wird als Zusatzlayer eingesetzt
- „Werfen“ Daten aus den feature maps mit wenig Signifikanz für das Ergebnis raus
- Max-Pooling: Nur der größte Wert eines Receptive Fields wird behalten
 - Kanten und Texturen bleiben erhalten
 - Rauschen wird unterdrückt
- Average-Pooling: Durchschnitt aus allen Werten im Receptive Field wird gebildet und behalten
 - Glättung und Szenenerkennung



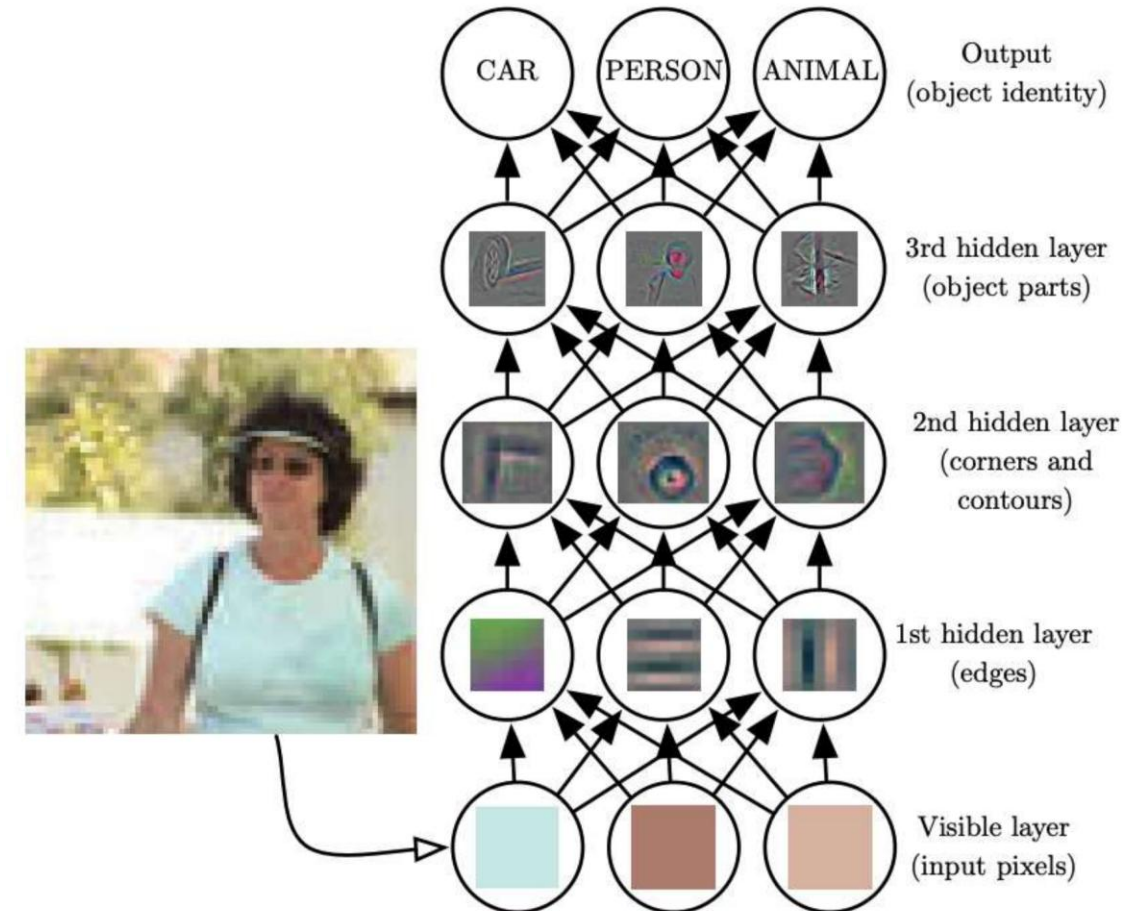
Grundlagen – Convolutional Neural Networks (CNNs)

- Tiefe neuronale Netze aus vielen CONV und Pooling-Layern können komplexe Bilder auswerten
- Beispiel unten: Bildsegmentierung
 - Jeder Pixel wird einer Klasse zugeordnet
 - Typische Anwendung in der Umgebungserkennung in der Robotik



Grundlagen – CNN-Funktionsprinzip

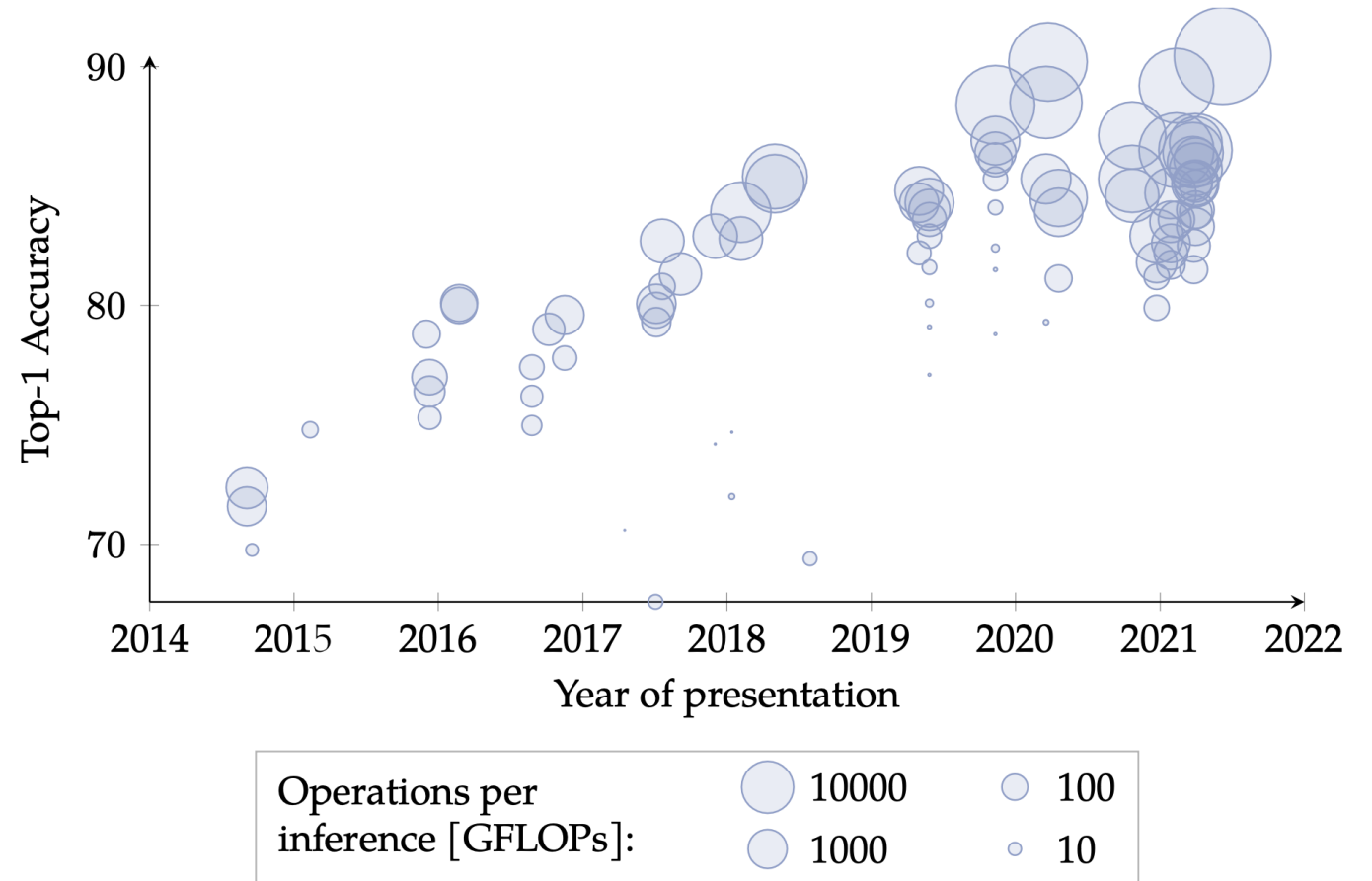
- Wie funktioniert das?
 - Filter in den ersten Layern erkennen Kanten (Low-Level Features)
 - Weitere Filter vereinen Kanten zu komplexeren Konturen
 - Die letzten Layer vereinen diese Konturen zu komplexen Objekten und Features
- Selbst ausprobieren:
<https://poloclub.github.io/cnn-explainer/>



Deep Learning, Goodfellow et. al.

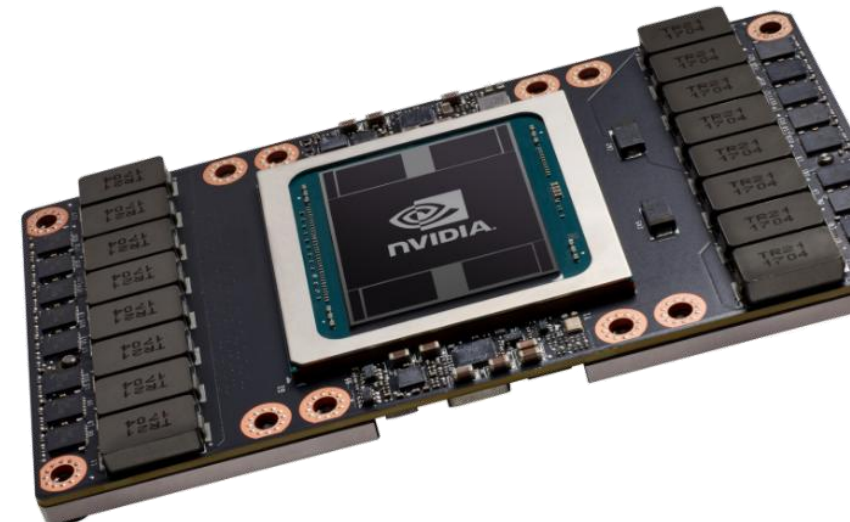
Neuronale Netze – Trends

- In den letzten Jahren sind NNs immer größer geworden
 - Mehr Gewichte / Parameter
 - komplexere Anwendungen
- Sie werden auch immer aufwendiger zu Berechnen
- Aber es gibt Lösungen
→ **Hardwarebeschleunigung**



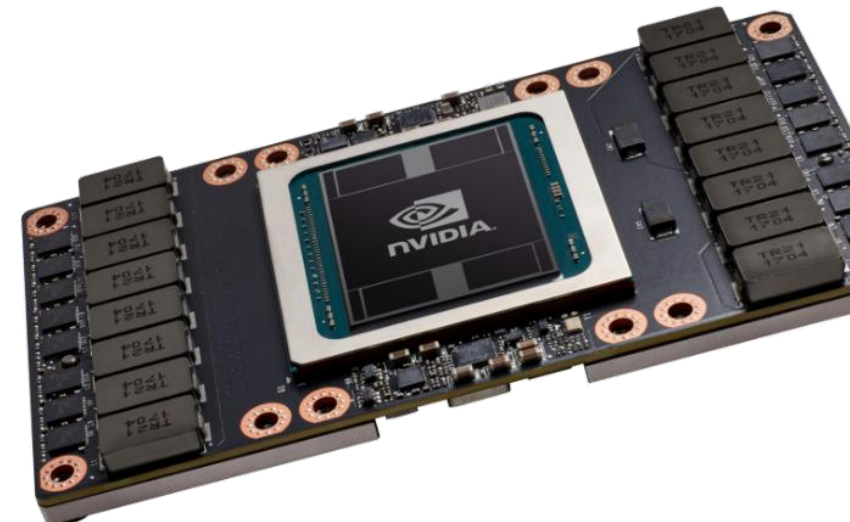
NN-Berechnung in HW

- Alle Arten von NNs können größtenteils mit MAC (Multiply-Accumulate) berechnet werden
 - Genauer gesagt mit Matrix-Matrix-Multiplikationen
- NN-Berechnung ist eine stark Datenfluss-getriebene Anwendung
 - Problem: CPUs können gut Anwendungen mit viel Kontrollfluss lösen
 - Spezialhardware fokussiert sich entsprechend auf die effiziente Ausführung des Datenflusses (MAC-Operationen)
 - Logik für Kontrollfluss wird fast komplett wegoptimiert
- GPUs eignen sich z.B. sehr gut für NN-Anwendungen auf Grund der vielen parallelen Recheneinheiten



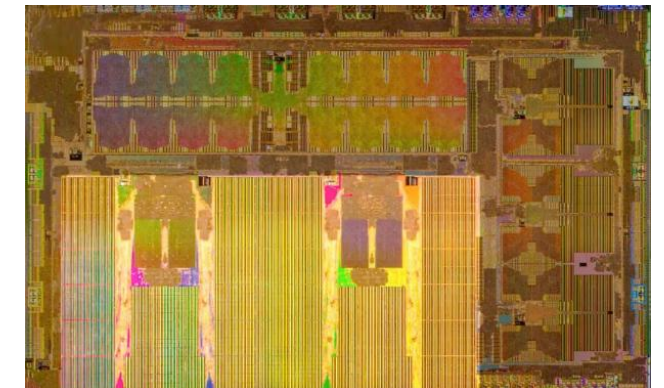
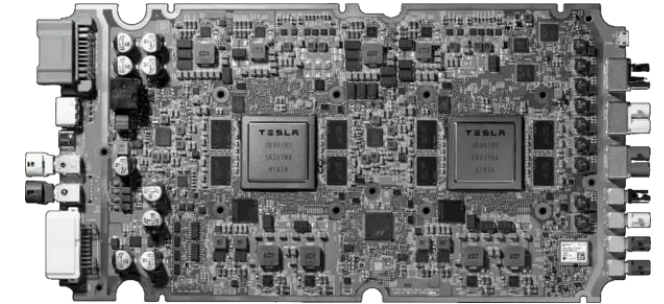
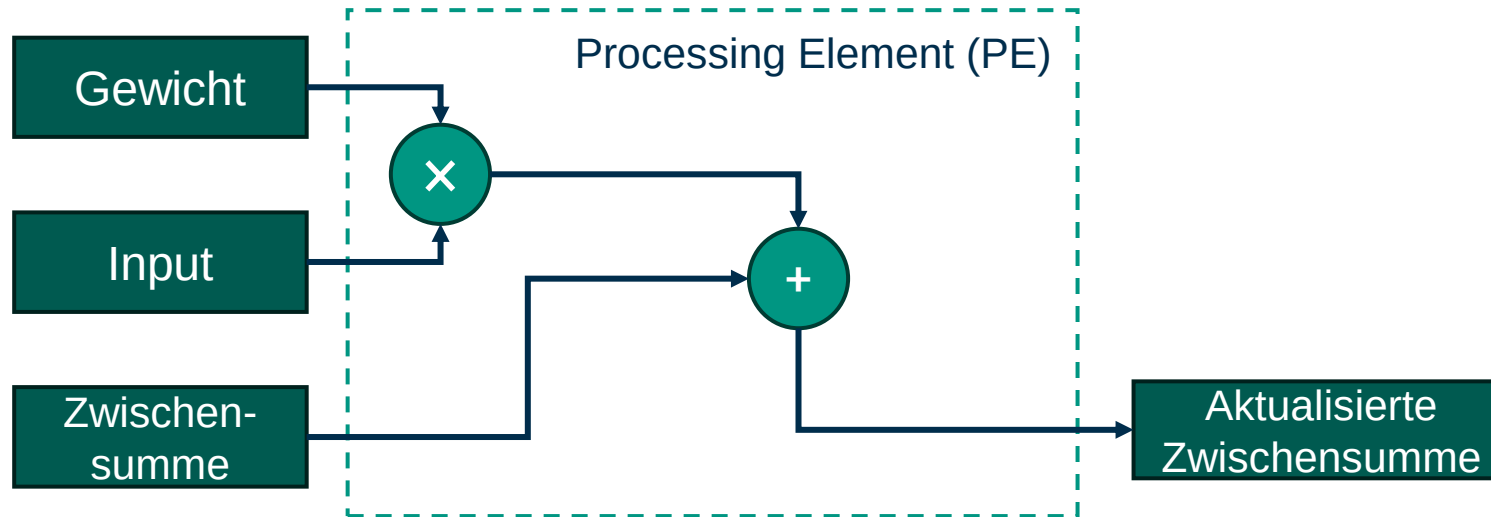
NN-Berechnung in HW – Was heißt effizient?

- Effizienz bei der Inferenz
 - Optimierung des Vorwärtsthroughs → Minimierung/Maximierung einer vorgegebenen Metrik bei der Berechnung der Ausgabe
- Verschiedene Metriken können herangezogen werden:
 - Latenz: Dauer für eine NN-Inferenz
 - Energieverbrauch pro NN-Inferenz
 - Durchsatz: NN-Inferenzen oder MAC-Operationen pro Sekunde
 - Chipfläche
 - Speicherbedarf
 - Modelgenauigkeit (Accuracy, MSE, usw...)



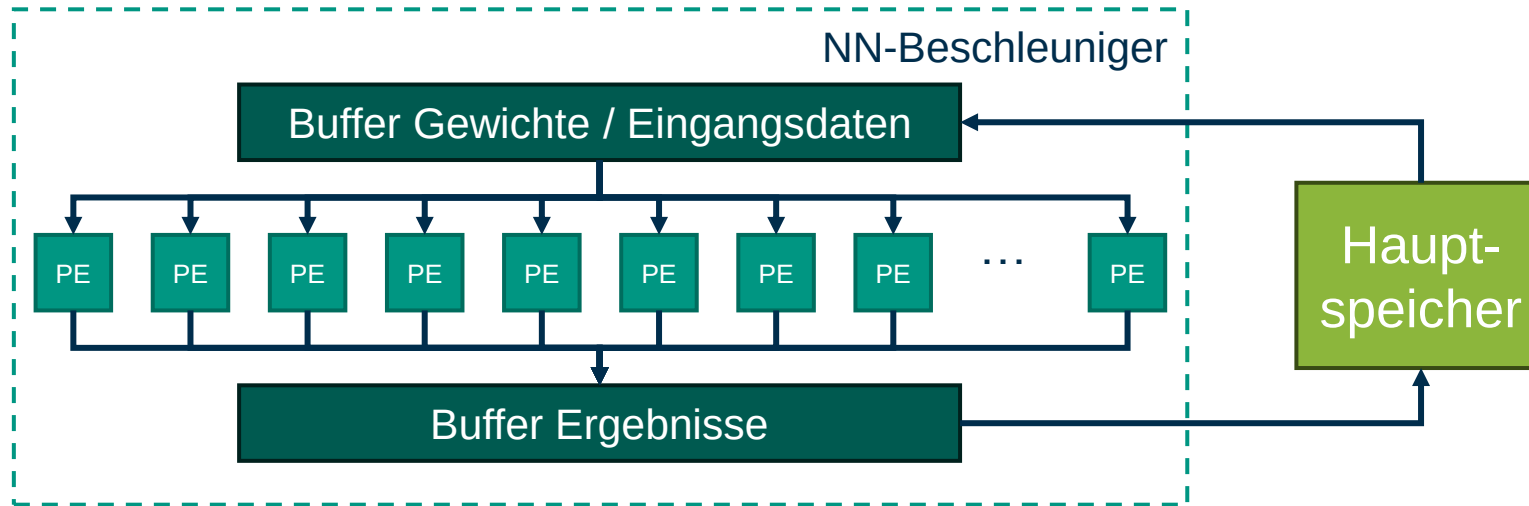
NN-Berechnung in HW – Spezialhardware

- Grundsätzlicher Aufbau
 - Speicher für Operanden und Zwischenergebnisse
 - MAC-Einheit: Multiplizier und Addierer mit Register
 - → Oft zusammengefasst als Processing Element (PE)

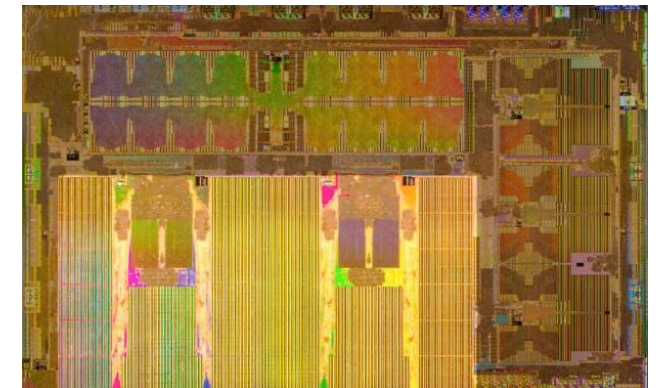
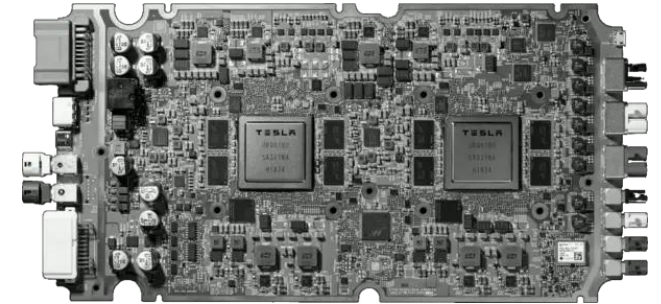


NN-Berechnung in HW – Spezialhardware

- Grundsätzlicher Aufbau: NN-Beschleuniger
 - Viele zusammengeschaltete PEs
 - Große globale Speicher



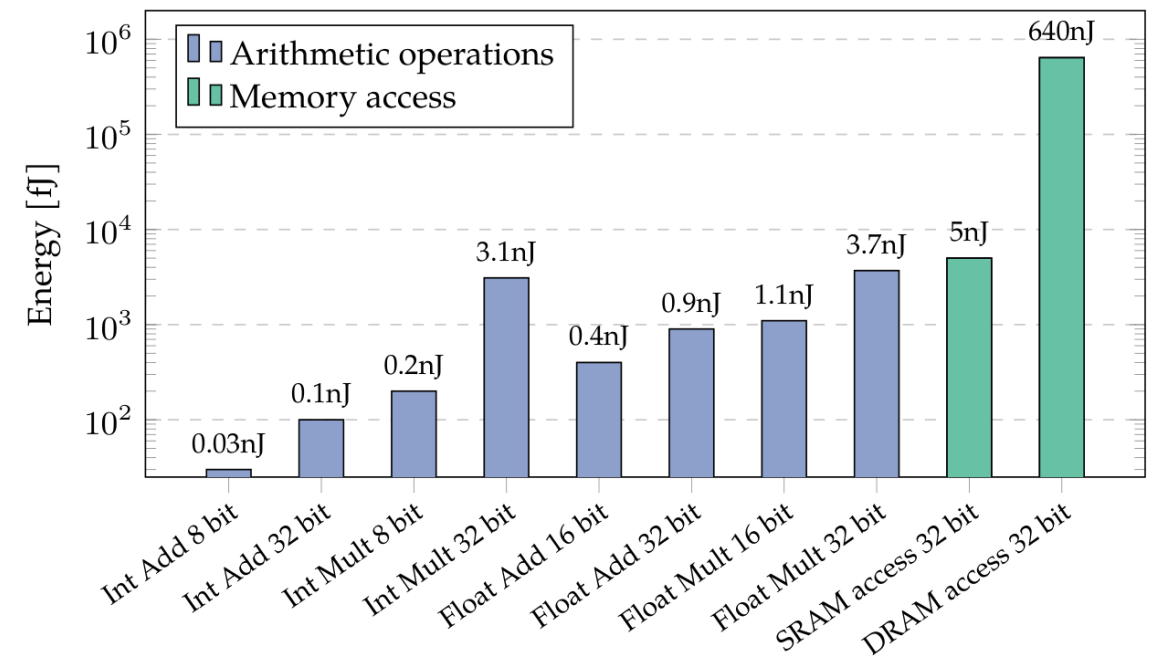
Problem: **Unzählige Speicherzugriffe!**
Bsp: AlexNet: 2.896 Millionen Zugriffe pro Inferenz



NN-Berechnung in HW – Spezialhardware

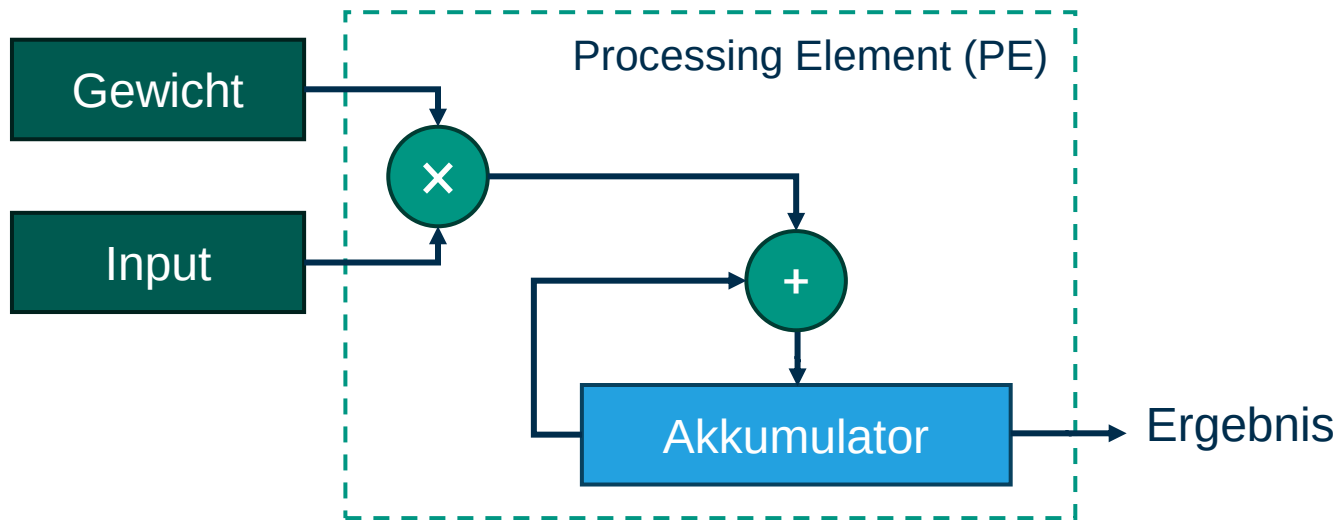
Problem: **Unzählige Speicherzugriffe!**
Bsp: AlexNet: 2.896 Millionen Zugriffe pro Inferenz

- Und Speicherzugriffe kosten viel Energie
- Beispiel: Ein 32-bit DRAM-Zugriff benötigt 200x mal Energie als eine 32-bit MAC-Operation
 - Evaluiert auf 45nm CMOS mit 0.9V Versorgungsspannung
 - Nach Horowitz et al., 2014

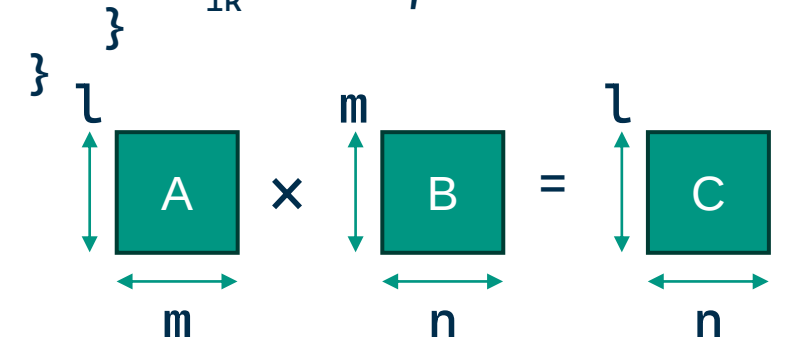


NN-Berechnung in HW – Datenflussoptimierung

- Die Matrix-Matrix-Multiplikation kann beliebig permutiert und verschiedene Operanden können wiederverwendet werden
- Datenfluss: Output-Stationary (Zwischenergebnis bleibt stationär)

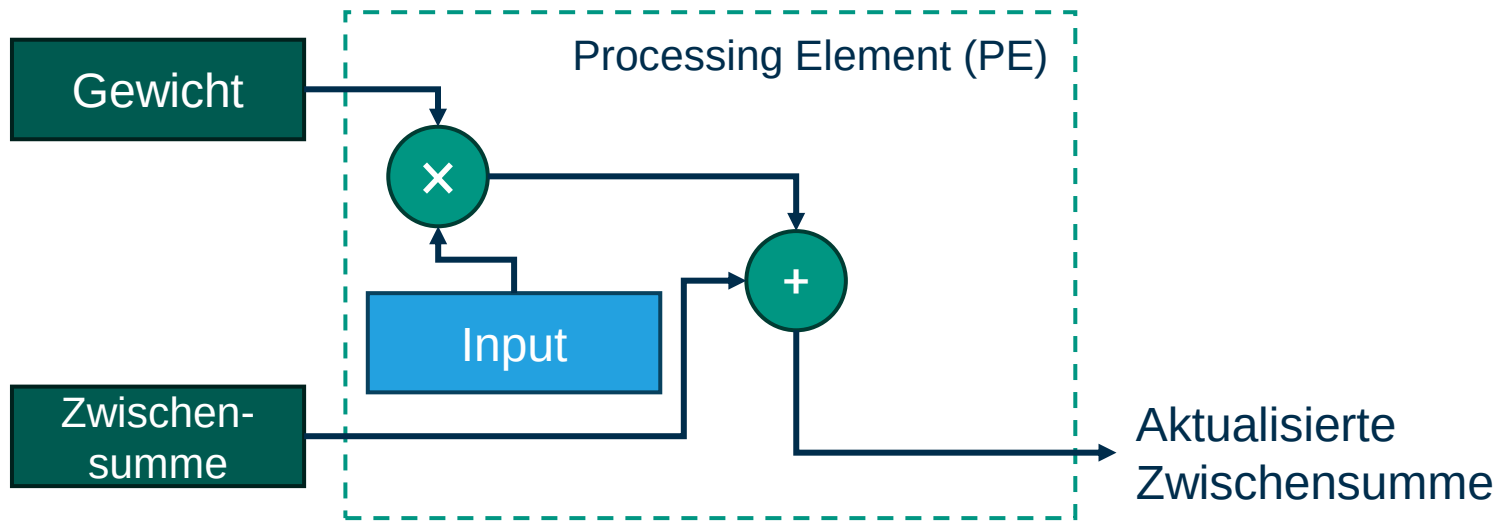


```
for (i=0; i<l; i++) {  
    for (k=0; k<n; k++) {  
        acc = 0;  
        for (j=0; j<m; j++) {  
            acc = acc + Aij * Bjk  
        }  
        Cik = acc;  
    }  
}
```



NN-Berechnung in HW – Datenflussoptimierung

- Die Matrix-Matrix-Multiplikation kann beliebig permutiert und verschiedene Operanden können wiederverwendet werden
- Datenfluss: Input-Stationary (Eingangsdatum bleibt stationär)

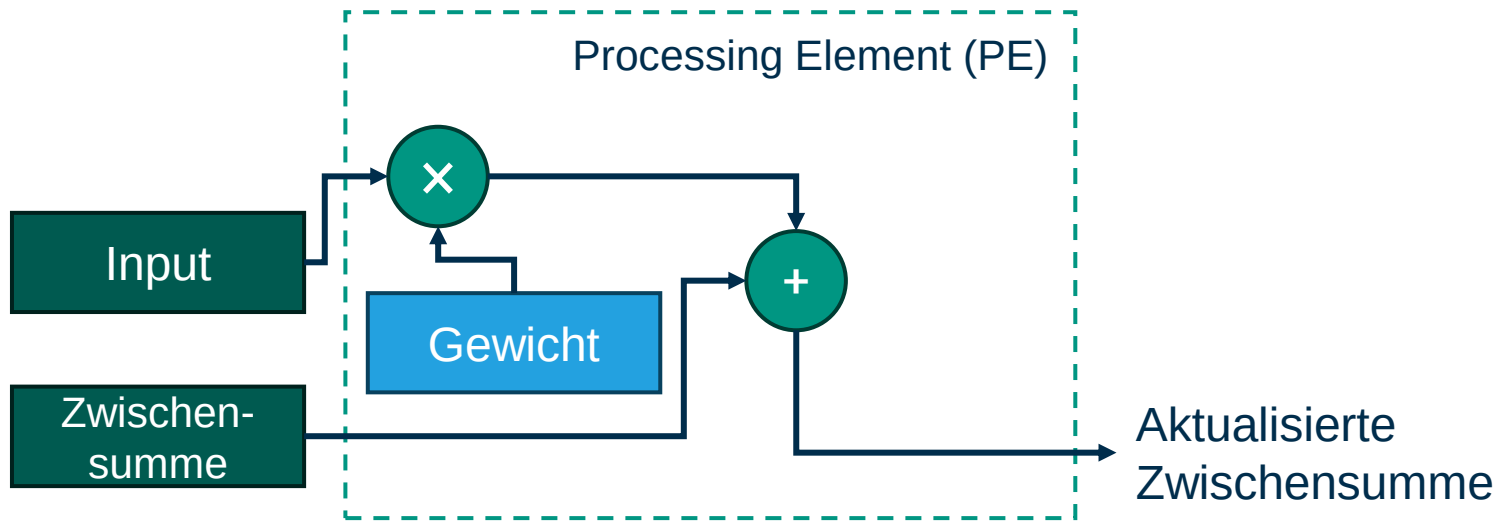


```
for (i=0; i<l; i++) {  
    for (j=0; j<m; j++) {  
        in = Aij  
        for (k=0; k<n; k++) {  
            Cik += Cik + in * Bjk  
        }  
    }  
}
```

$$\begin{matrix} l \\ \updownarrow \\ \text{A} \\ \leftarrow m \end{matrix} \times \begin{matrix} m \\ \updownarrow \\ \text{B} \\ \leftarrow n \end{matrix} = \begin{matrix} l \\ \updownarrow \\ \text{C} \\ \leftarrow n \end{matrix}$$

NN-Berechnung in HW – Datenflussoptimierung

- Die Matrix-Matrix-Multiplikation kann beliebig permutiert und verschiedene Operanden können wiederverwendet werden
- Datenfluss: Weight-Stationary (Gewicht bleibt stationär)

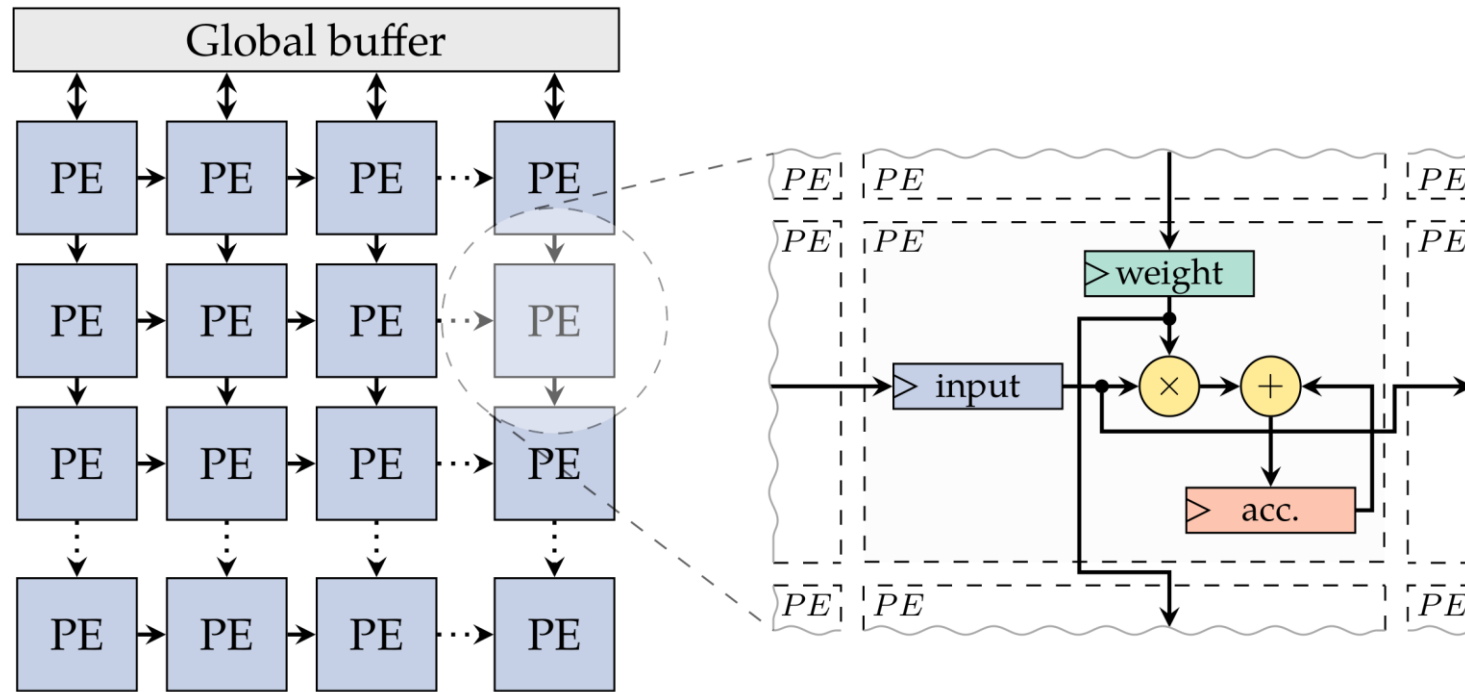


```
for (j=0; i<l; i++) {  
  for (k=0; k<n; k++) {  
     $w = B_{ik}$   
    for (i=0; i<l; i++) {  
       $C_{ik} += C_{ik} + A_{ij} * w$   
    }  
  }  
}
```

$$\begin{matrix} l \\ \updownarrow \\ \text{A} \\ \text{m} \end{matrix} \times \begin{matrix} m \\ \updownarrow \\ \text{B} \\ n \end{matrix} = \begin{matrix} l \\ \updownarrow \\ \text{C} \\ n \end{matrix}$$

NN-Berechnung in HW – Systolische Arrays

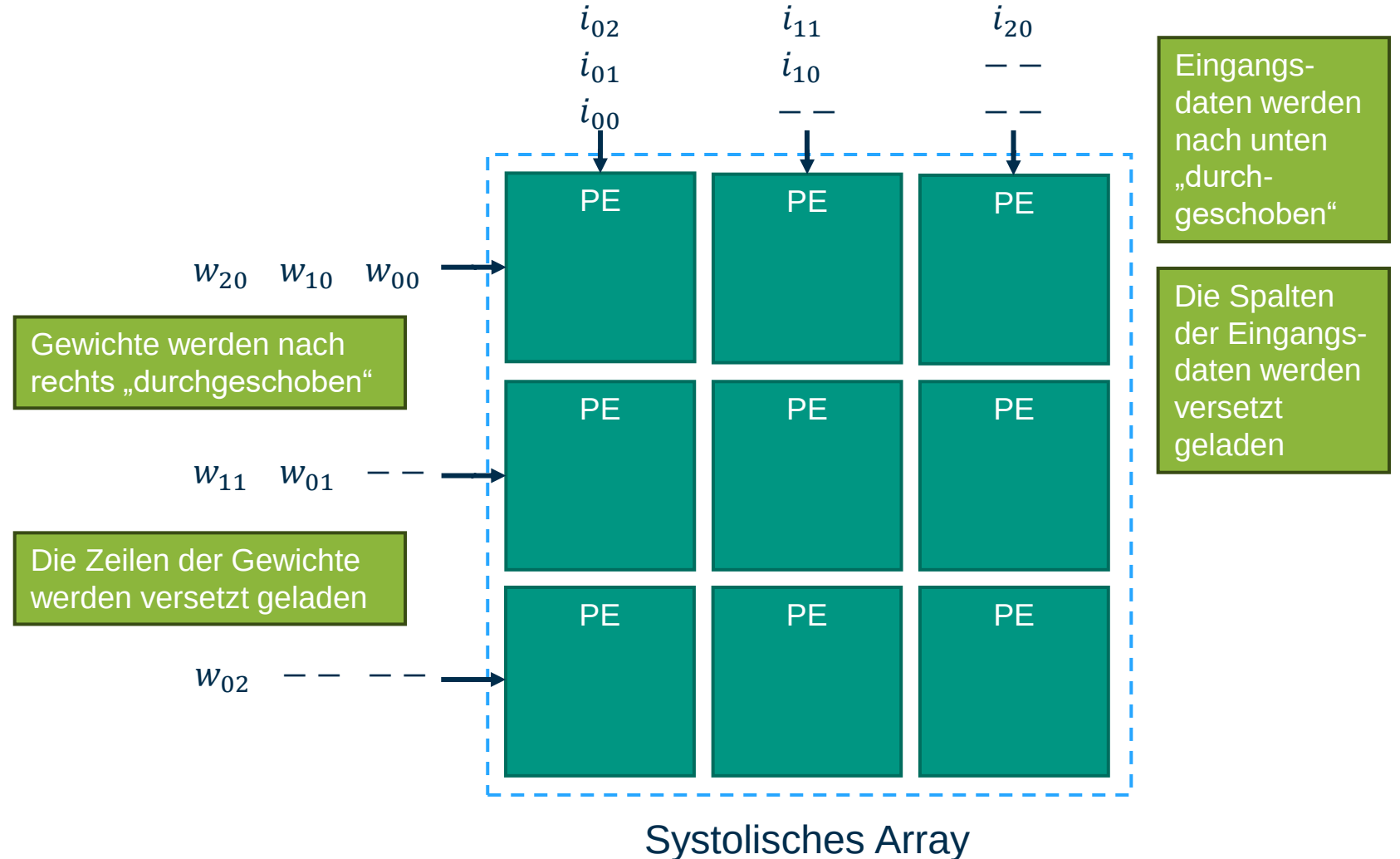
- Datenwiederverwendung wird in systolischen Arrays maximiert



Output-Stationary systolic Array nach Sze et al., 2020

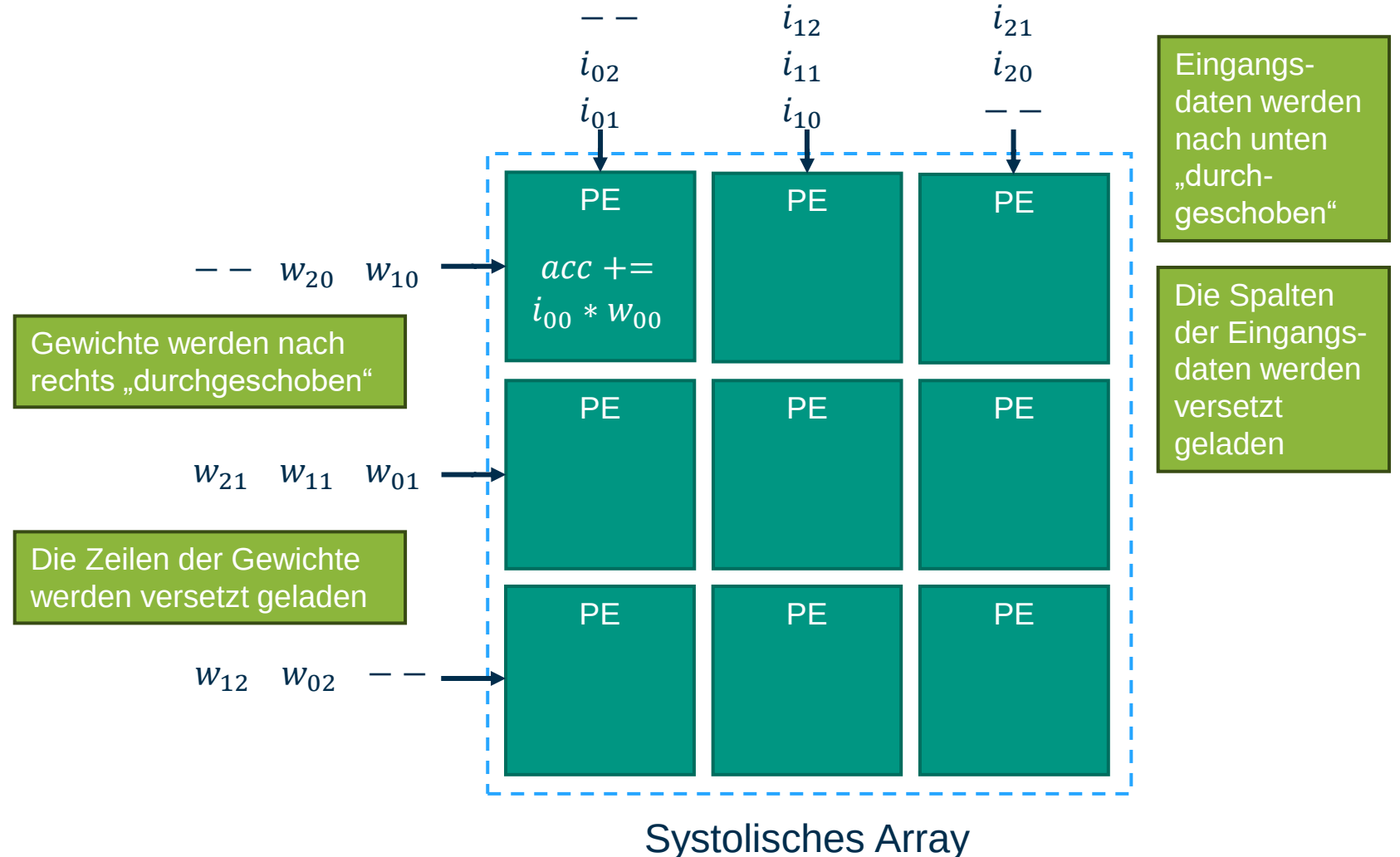
NN-Berechnung in HW – Systolische Arrays

- Datenwiederverwendung wird in systolischen Arrays maximiert
- Beispiel 3x3 Array mit 3x3 Matrizen
- Funktionsprinzip: **Schritt 1**



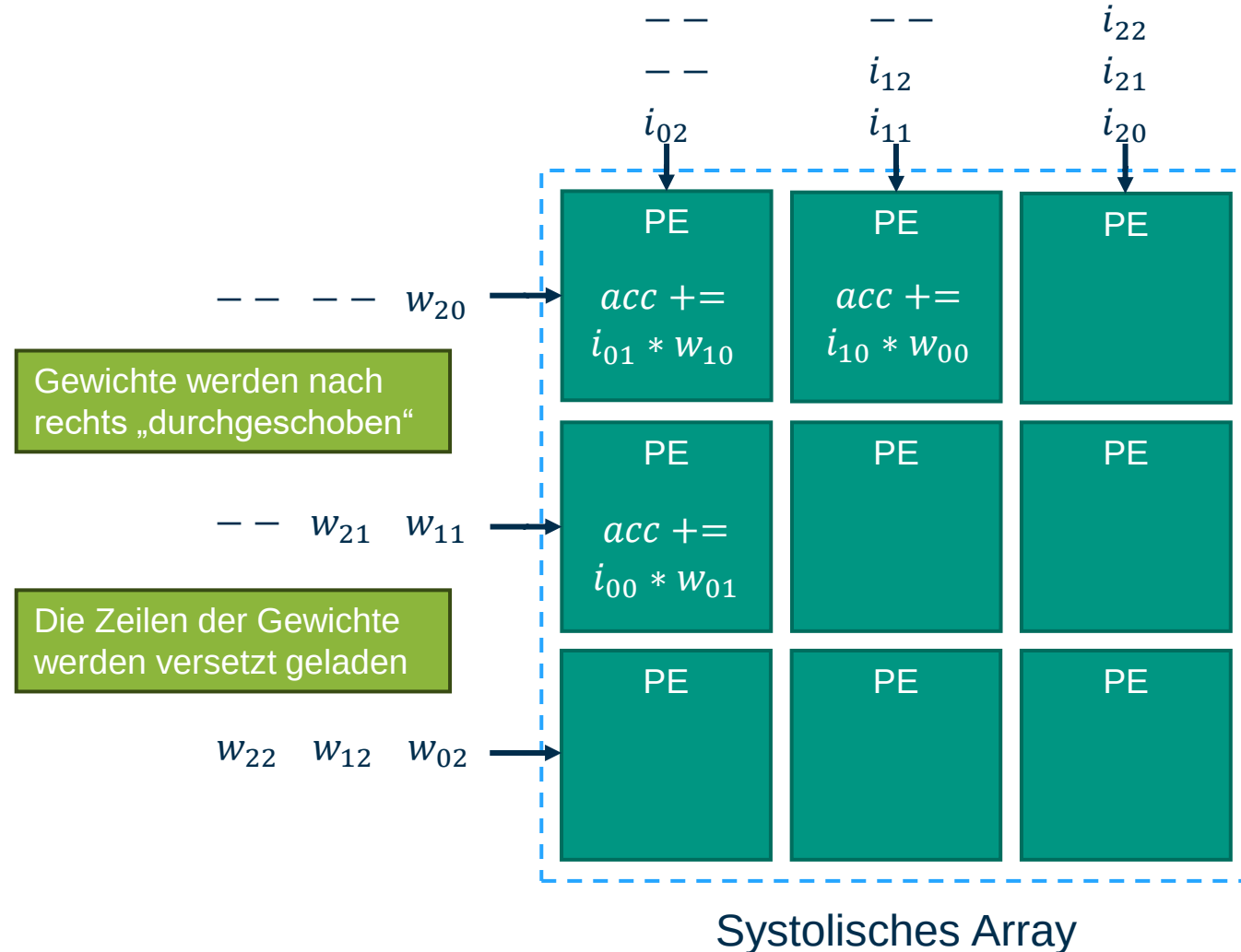
NN-Berechnung in HW – Systolische Arrays

- Datenwiederverwendung wird in systolischen Arrays maximiert
- Beispiel 3x3 Array mit 3x3 Matrizen
- Funktionsprinzip: **Schritt 2**



NN-Berechnung in HW – Systolische Arrays

- Datenwiederverwendung wird in systolischen Arrays maximiert
- Beispiel 3x3 Array mit 3x3 Matrizen
- Funktionsprinzip: **Schritt 3**



Eingangsdaten werden nach unten „durchgeschoben“

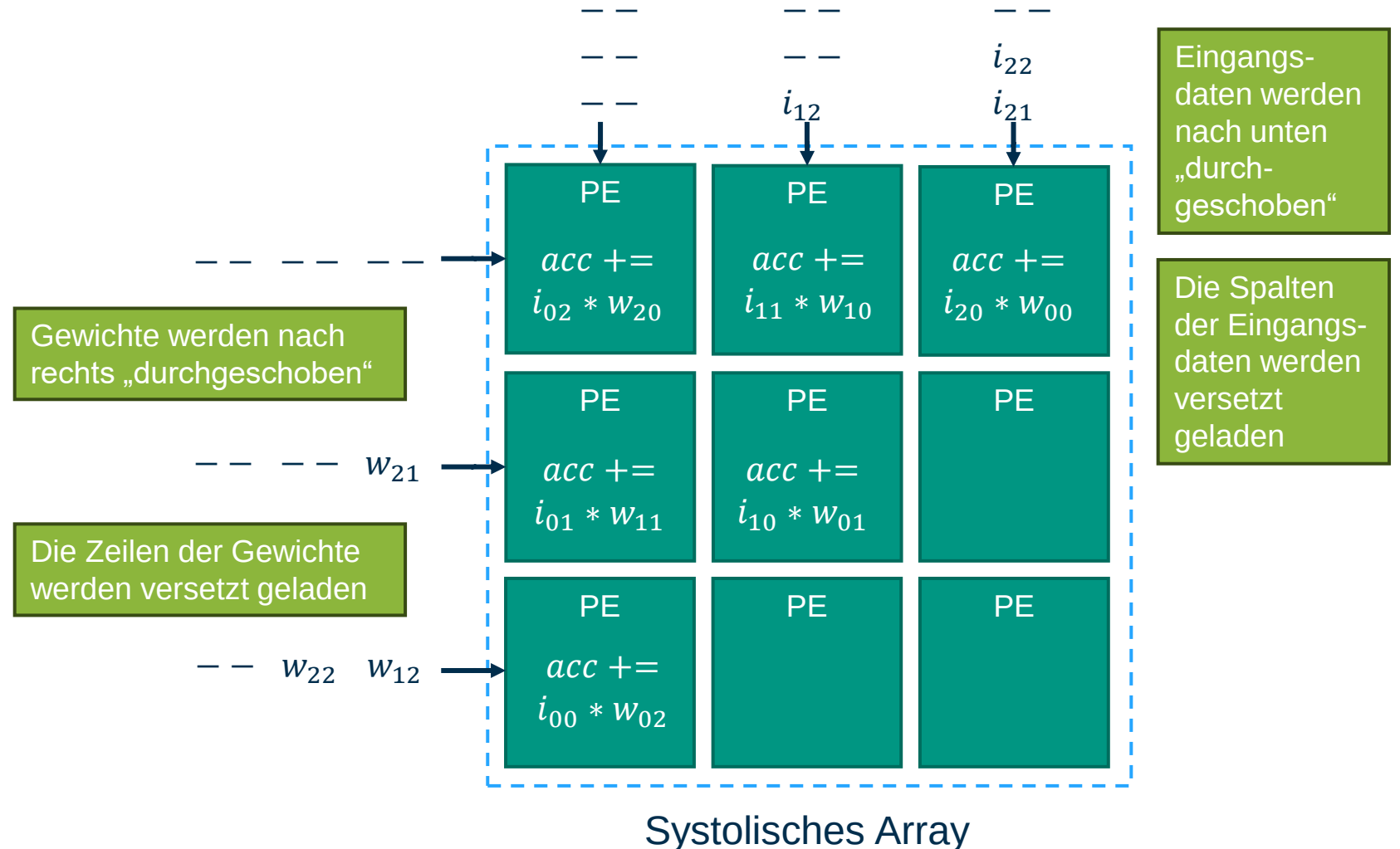
Die Spalten der Eingangsdaten werden versetzt geladen

Gewichte werden nach rechts „durchgeschoben“

Die Zeilen der Gewichte werden versetzt geladen

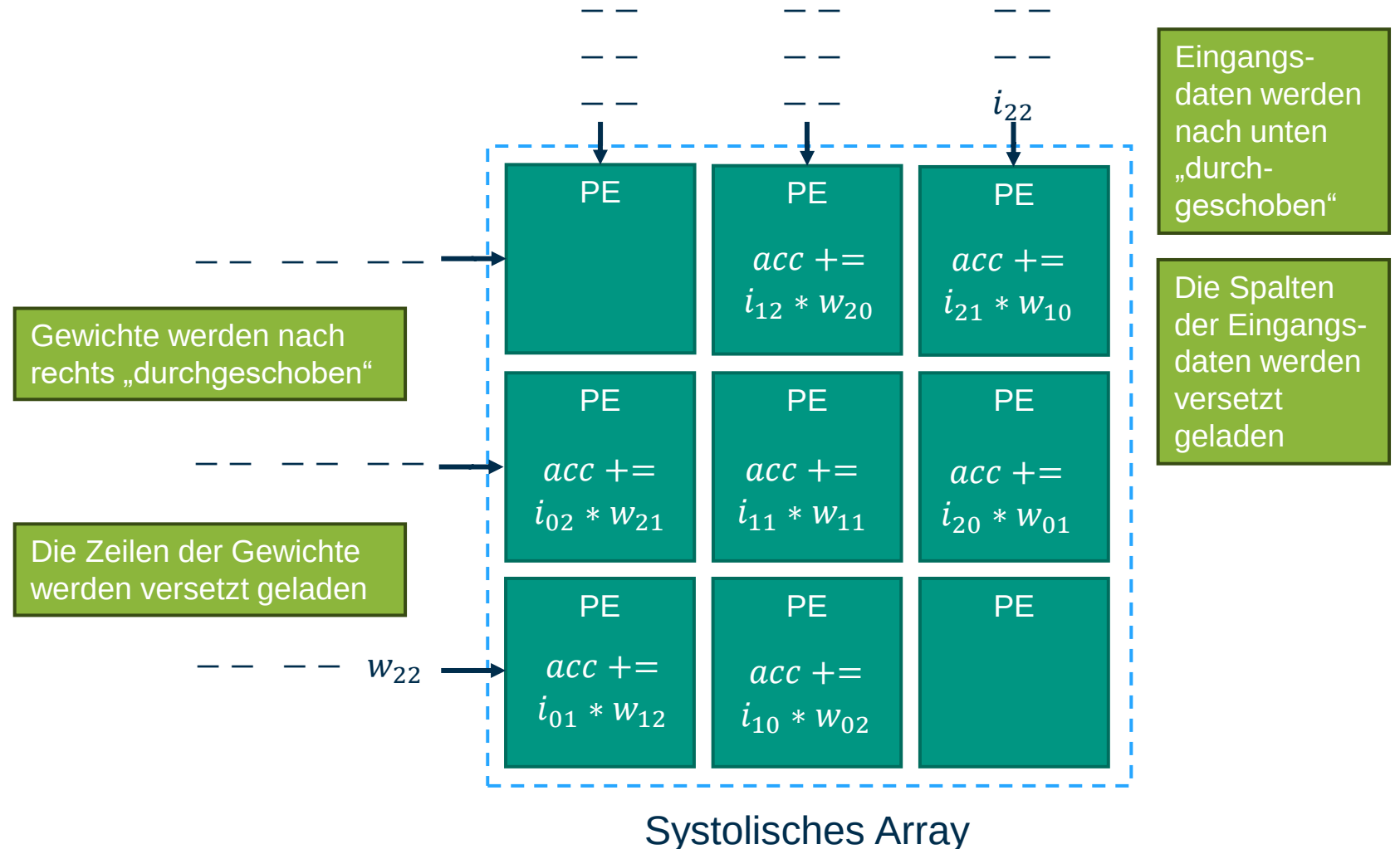
NN-Berechnung in HW – Systolische Arrays

- Datenwiederverwendung wird in systolischen Arrays maximiert
- Beispiel 3x3 Array mit 3x3 Matrizen
- Funktionsprinzip: **Schritt 4**



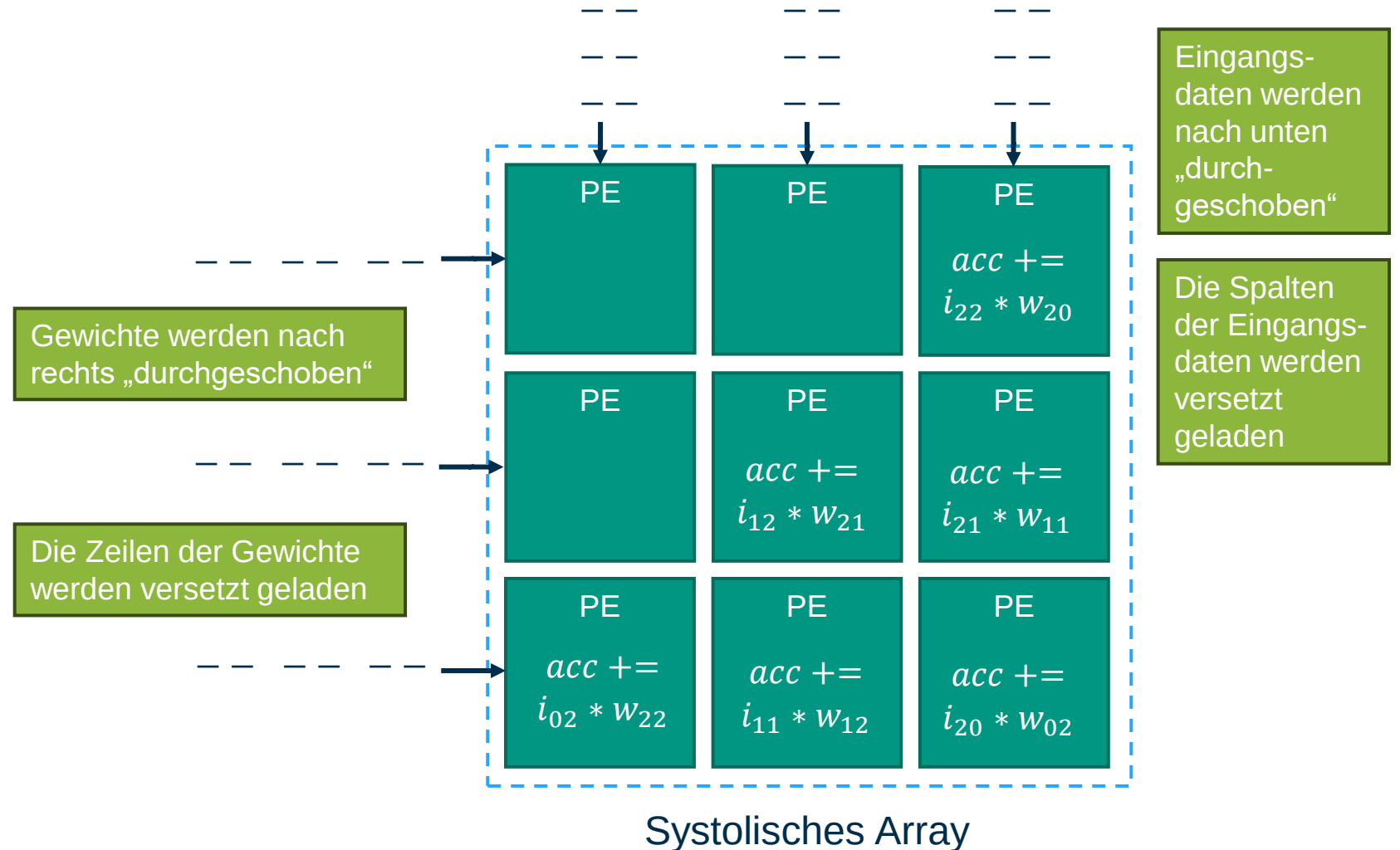
NN-Berechnung in HW – Systolische Arrays

- Datenwieder-
verwendung wird in
systolischen Arrays
maximiert
- Beispiel 3x3 Array mit
3x3 Matrizen
- Funktionsprinzip:
Schritt 5



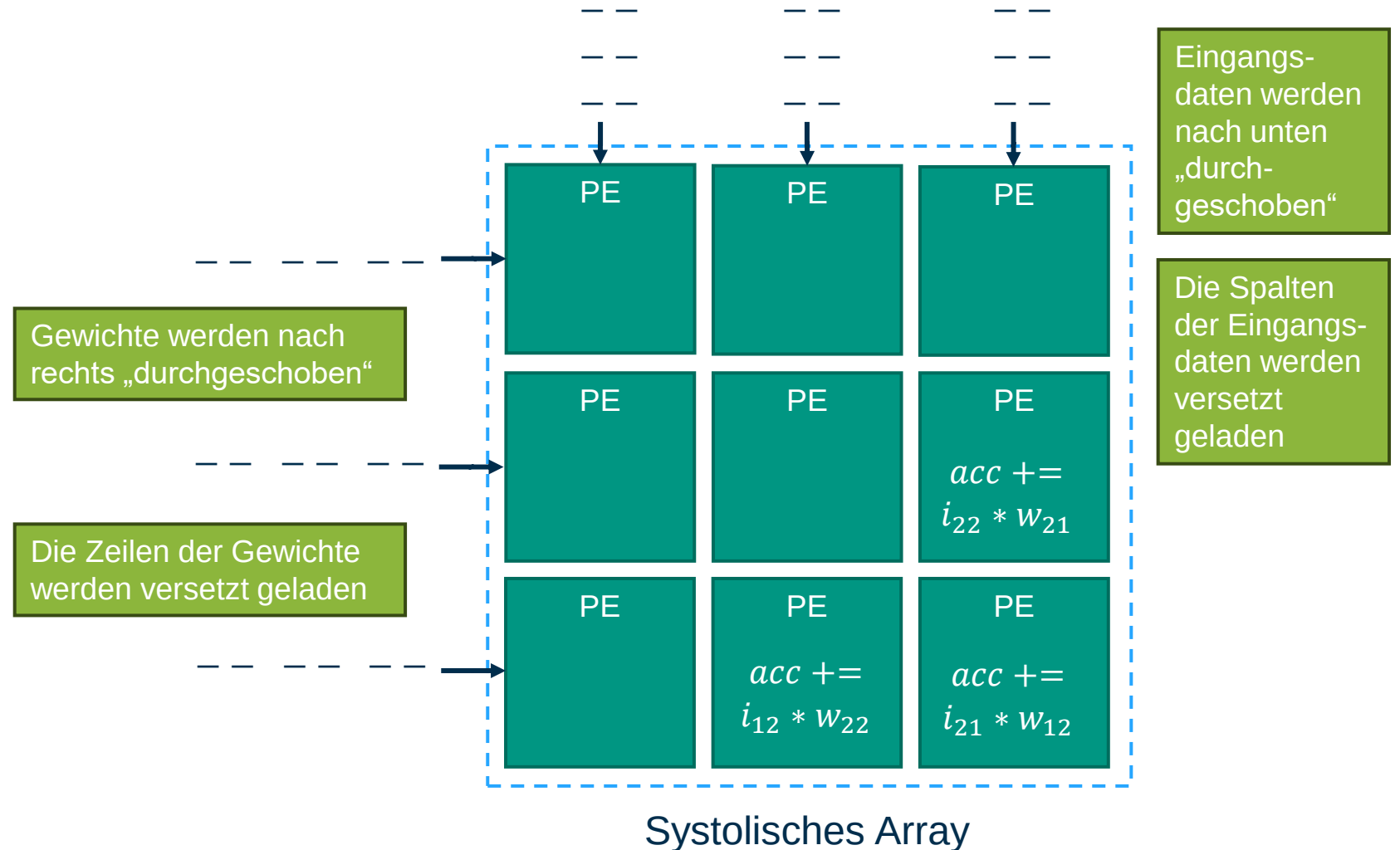
NN-Berechnung in HW – Systolische Arrays

- Datenwiederverwendung wird in systolischen Arrays maximiert
- Beispiel 3x3 Array mit 3x3 Matrizen
- Funktionsprinzip: **Schritt 6**



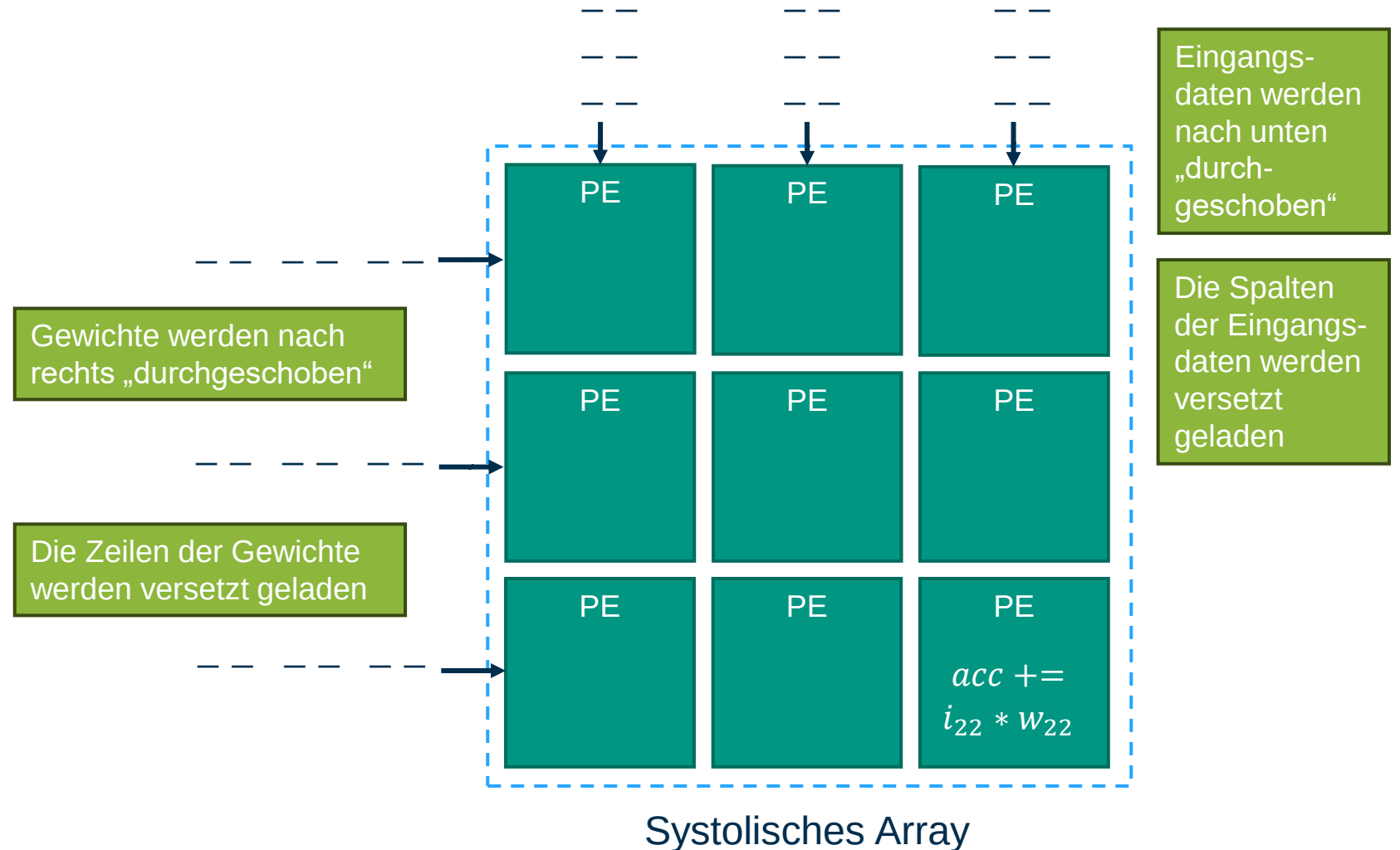
NN-Berechnung in HW – Systolische Arrays

- Datenwiederverwendung wird in systolischen Arrays maximiert
- Beispiel 3x3 Array mit 3x3 Matrizen
- Funktionsprinzip: **Schritt 7**



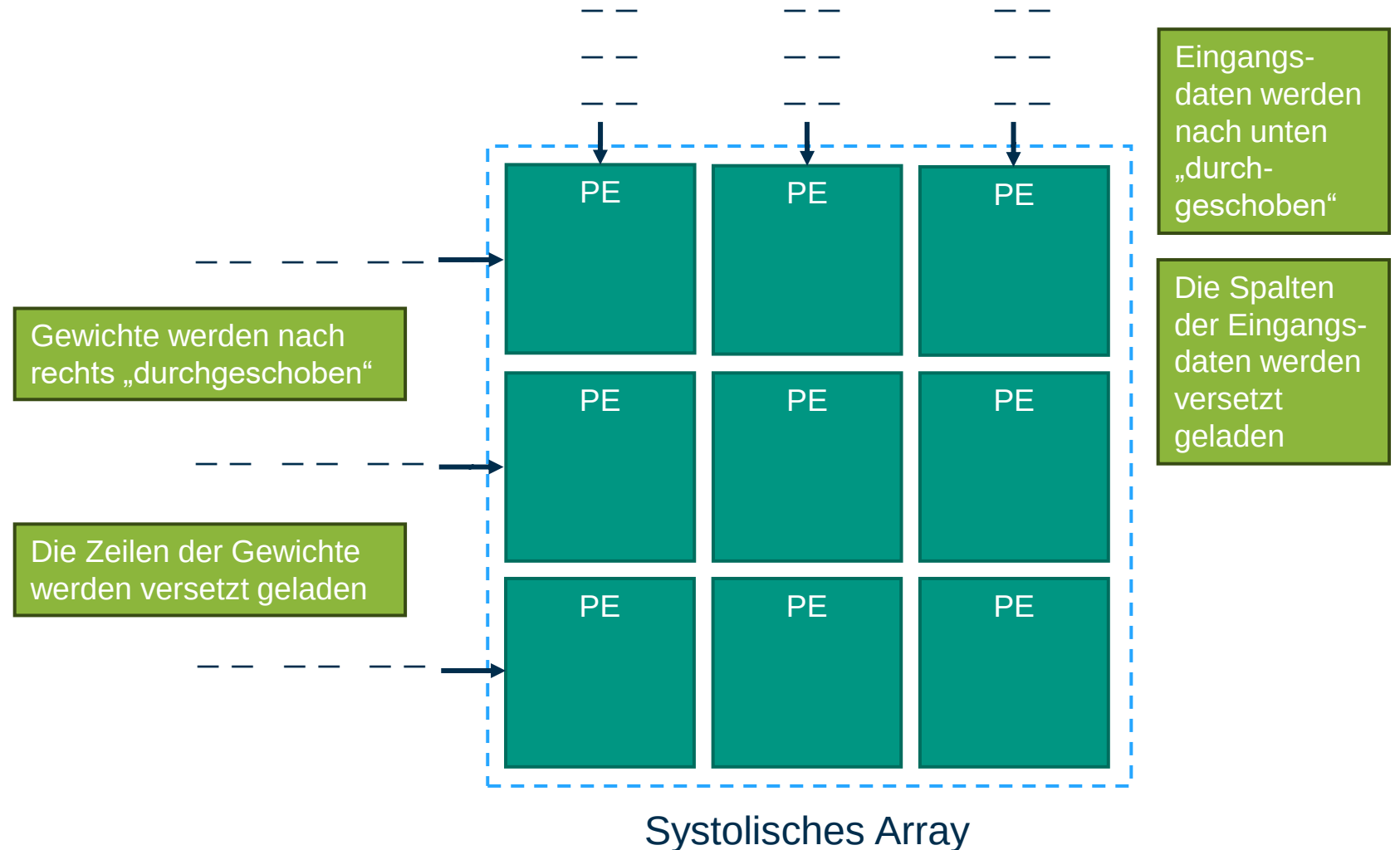
NN-Berechnung in HW – Systolische Arrays

- Datenwiederverwendung wird in systolischen Arrays maximiert
- Beispiel 3x3 Array mit 3x3 Matrizen
- Funktionsprinzip: **Schritt 8**



NN-Berechnung in HW – Systolische Arrays

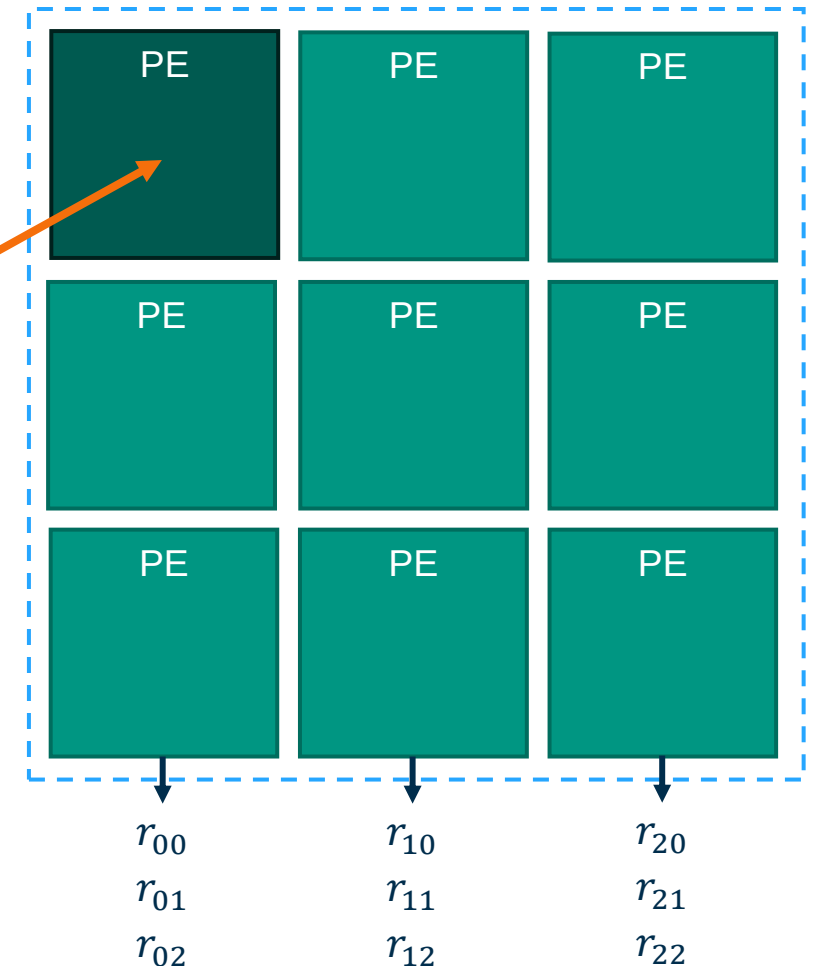
- Datenwiederverwendung wird in systolischen Arrays maximiert
- Beispiel 3x3 Array mit 3x3 Matrizen
- Funktionsprinzip: **Schritt 9 – Fertig**



NN-Berechnung in HW – Systolische Arrays

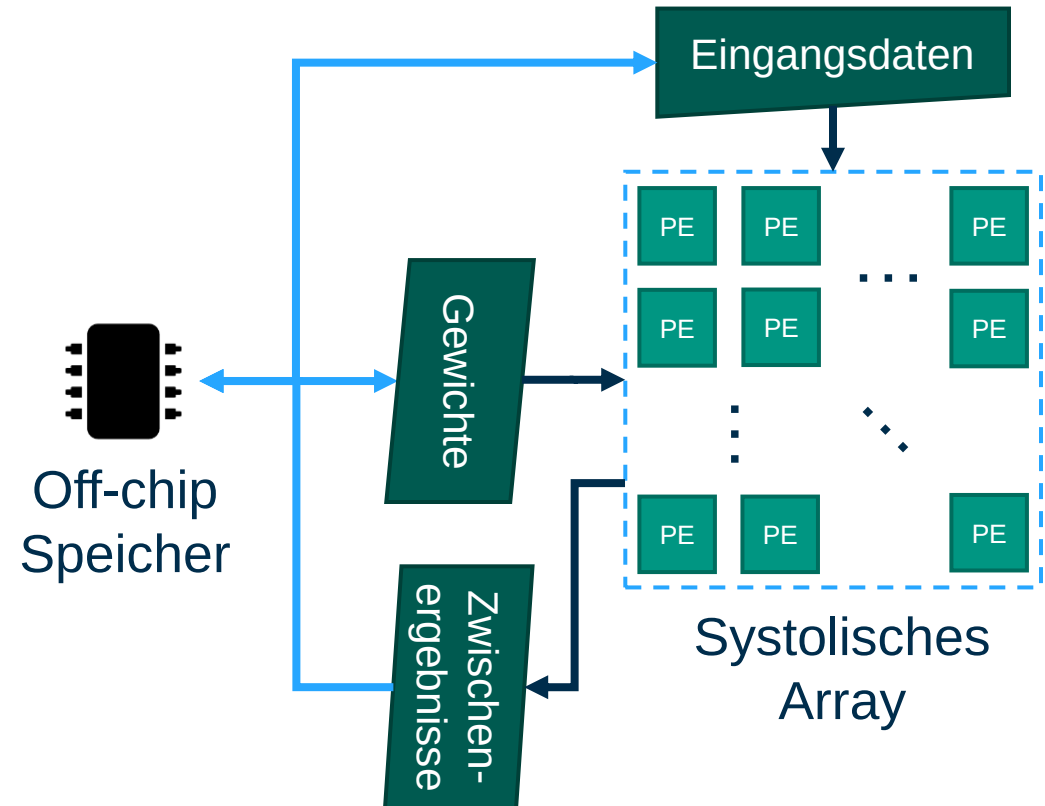
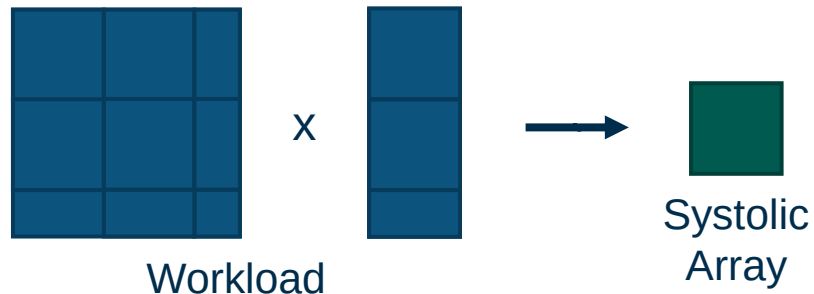
- Beispiel 3x3 Array mit 3x3 Matrizen
- Was wurde gerechnet?
 - In jedem PE steht ein Ergebnis der Matrix-Matrix-Multiplikation
 - Diese können in einem nächsten Schritt z.B. nach unten rausgelesen werden

$$acc = i_{00} * w_{00} + \\ i_{01} * w_{10} + \\ i_{02} * w_{20}$$



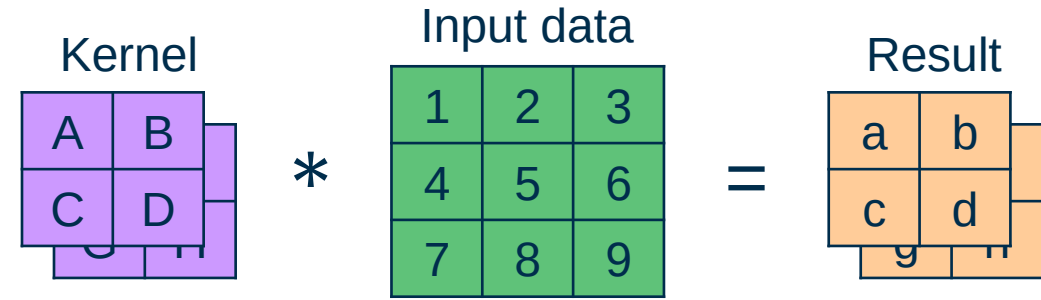
NN-Berechnung in HW – Systolische Arrays

- Größere Eingangsmatrizen werden aufgeteilt und die Zwischen-ergebnisse aufsummiert
- Die Operanden und Ergebnisse werden zwischengespeichert und ggf. auf externe Speicher ausgelagert

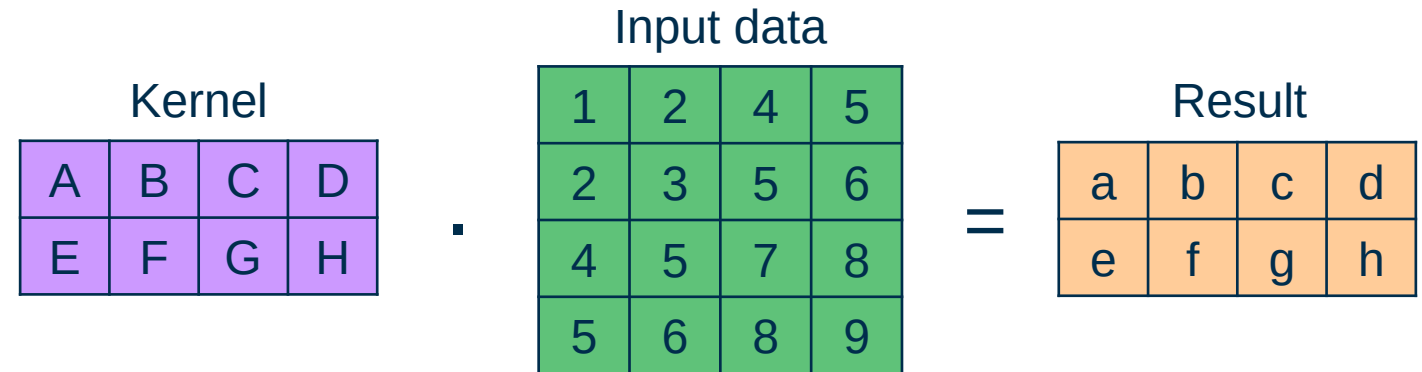


CNN-Berechnung in HW – Systolische Arrays

- Wie können CNNs berechnet werden?
- Erinnerung: Operanden der CNNs sind keine zwei-dimensionalen Matrizen sondern 4D-Tensoren
- Mit einer Umformung (im2col, image to column) können diese in 2D-Matrizen transformiert werden

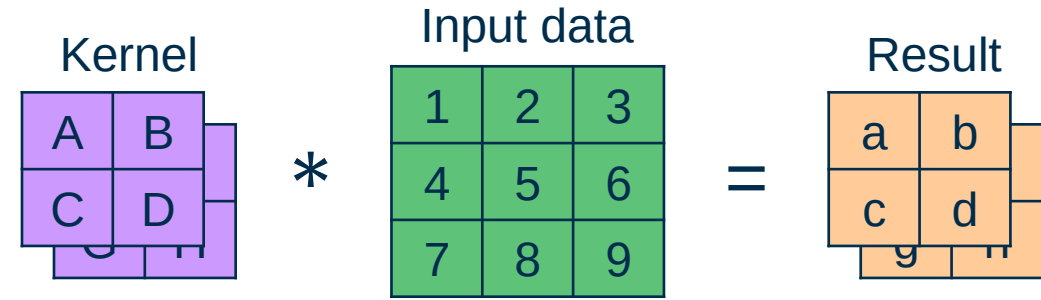


↓ im2col

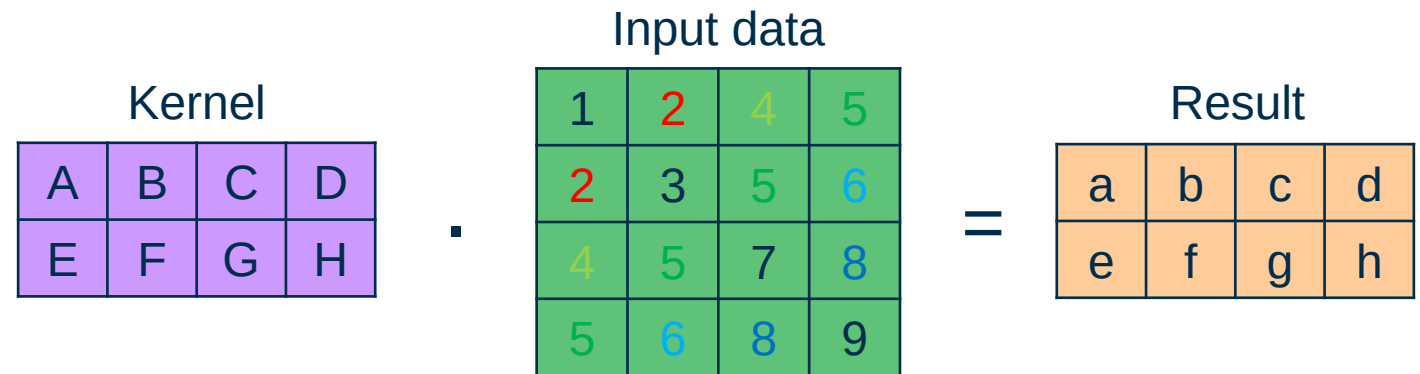


CNN-Berechnung in HW – Systolische Arrays

- Wie können CNNs berechnet werden?
- Mit einer Umformung (im2col) können diese in 2D-Matrizen transformiert werden
- Problem: **Datenduplikation**

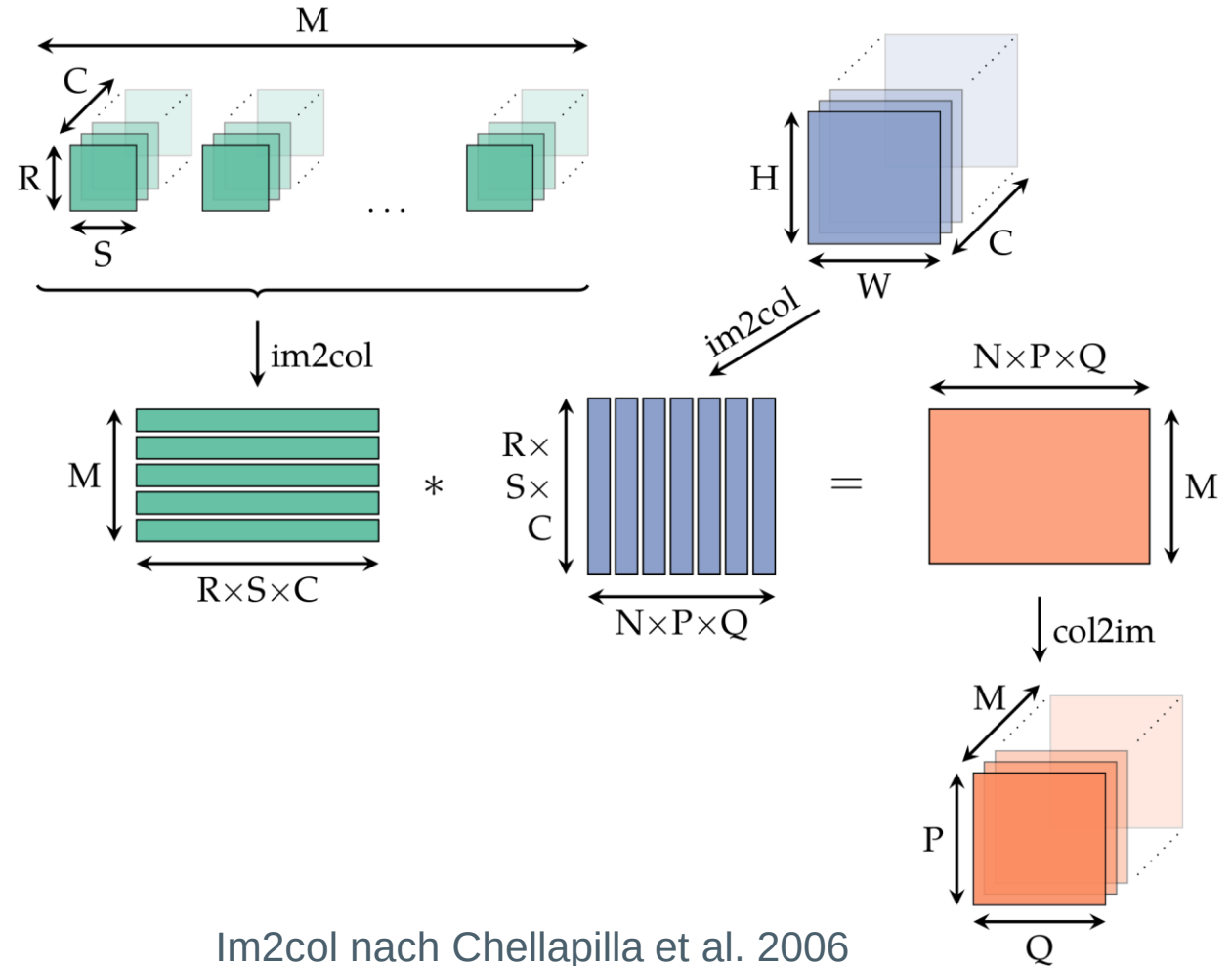


↓ im2col



CNN-Berechnung in HW – Systolische Arrays

- Wie können CNNs berechnet werden?
- Mit einer Umformung (im2col) können diese in 2D-Matrizen transformiert werden
- Problem: **Datenduplikation**
- Aber bei Tensoren mit vier Dimensionen weniger kritisch
- Effiziente Durchführung der Faltung als Matrixmultiplikation





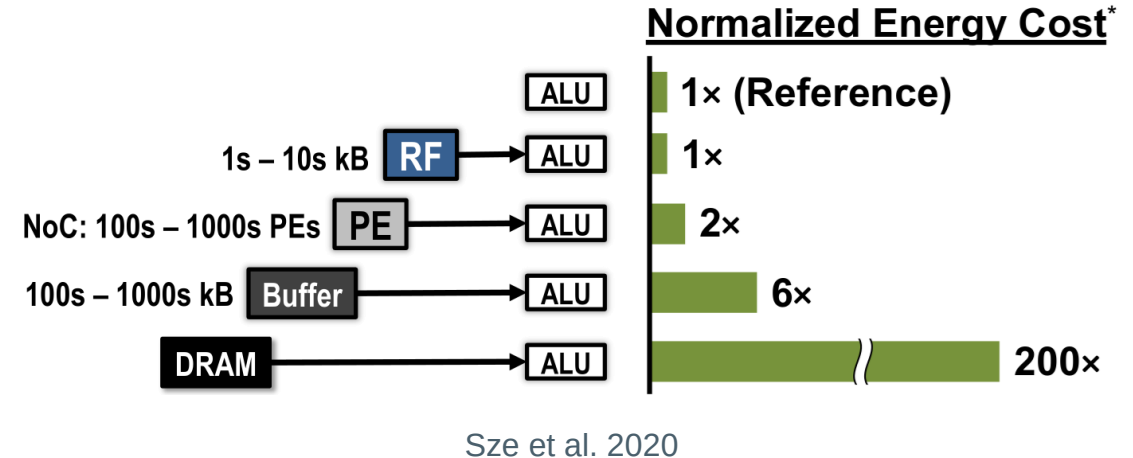
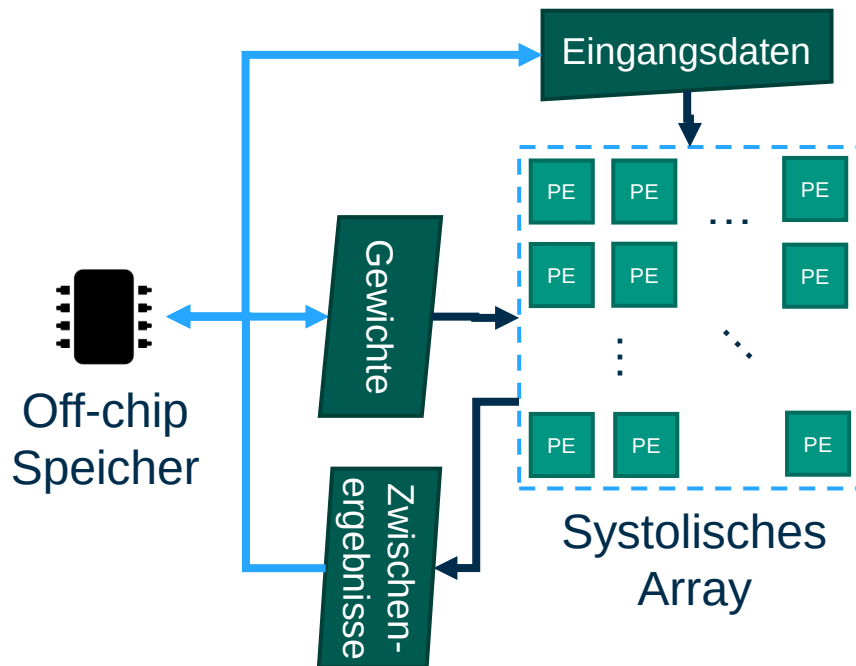
**Welche der folgenden Aussagen
über systolische Arrays sind
korrekt?**



Warum sind CPUs bei der Berechnung neuronaler Netze im Allgemeinen weniger effizient als spezialisierte Hardware?

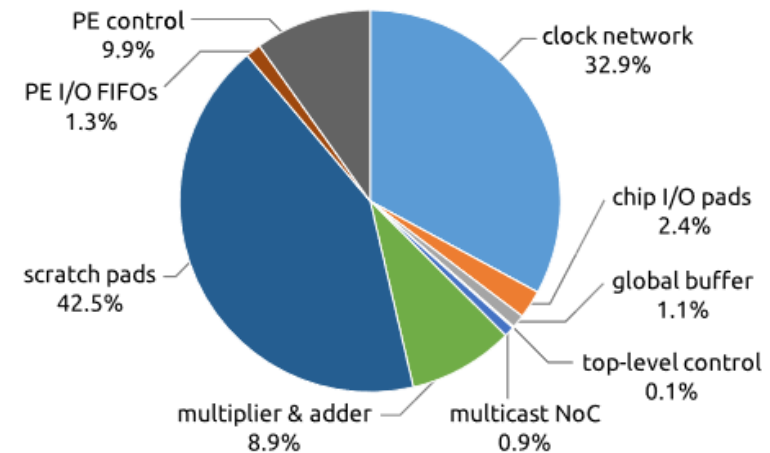
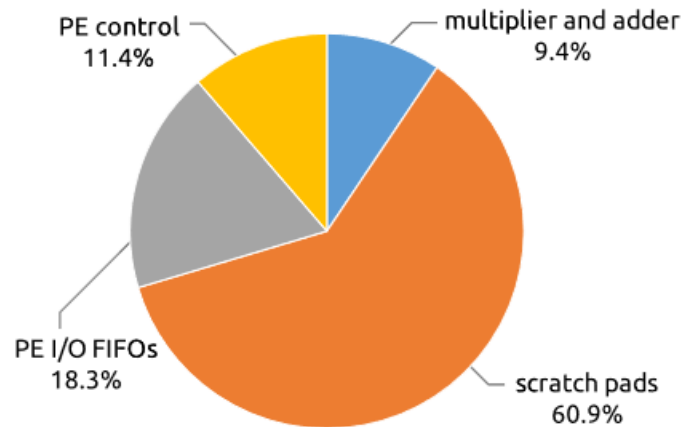
NN-Berechnung in HW – Herausforderungen

- Modelle werden immer größer um komplexere Aufgaben zu lösen
 - Parameter und Zwischenergebnisse müssen irgendwo gespeichert werden
 - externe Speicher sind zwar günstig, brauchen aber viel Energie und Fläche



NN-Berechnung in HW – Herausforderungen

- externe Speicher sind zwar günstig, brauchen aber viel Energie und Fläche
- Beispielauswertung: Scratch-Pads (lokale Speicher) brauchen 60,9% der Gesamtenergie und 42,5% der Chip-Fläche



Chen et al. 2017

NN-Berechnung in HW – Speicherhierarchie

- Konsequenz: Speicherhierarchien werden eingesetzt
 - Energieeffizientere lokale Speicher für „aktuell“ benötigte Daten
 - Große Off-Chip Speicher für demnächst benötigte Daten

